

ROBOREWARD: GENERAL-PURPOSE VISION-LANGUAGE REWARD MODELS FOR ROBOTICS

Tony Lee*
Stanford University

Andrew Wagenmaker*
UC Berkeley

Karl Pertsch*
UC Berkeley
Stanford University

Percy Liang
Stanford University

Sergey Levine
UC Berkeley

Chelsea Finn
Stanford University

ABSTRACT

A **well-designed reward** is critical for effective reinforcement learning-based policy improvement. In real-world robotics, obtaining such rewards typically requires either labor-intensive human labeling or brittle, handcrafted objectives. **Vision-language models (VLMs)** have shown promise as automatic reward models, yet their effectiveness on real robot tasks is poorly understood. In this work, we aim to close this gap by introducing (1) **RoboReward**, a **robotics reward dataset and benchmark** built on large-scale real-robot corpora from Open X-Embodiment (OXE) and RoboArena, and (2) **vision-language reward models** trained on this dataset (RoboReward 4B/8B). Because OXE is **success-heavy and lacks failure examples**, we propose a **negative examples data augmentation pipeline** that generates calibrated *negatives* and *near-misses* via counterfactual relabeling of successful episodes and temporal clipping to create partial-progress outcomes from the same videos. Using this framework, we build a large training and evaluation dataset spanning diverse tasks and embodiments to test whether **state-of-the-art VLMs can reliably provide rewards for robot learning**. Our evaluation of open and proprietary VLMs finds that **no model excels across tasks, highlighting substantial room for improvement**. We then train **general-purpose 4B- and 8B-parameter models that outperform much larger VLMs in assigning rewards for short-horizon robotic tasks**. Finally, we deploy the 8B model in real-robot reinforcement learning and find that it improves policy learning over Gemini Robotics-ER 1.5 while narrowing the gap to RL training with human-provided rewards. We release the full dataset, trained reward models, and evaluation suite on our website to advance the development of general-purpose reward models in robotics: <https://crfm.stanford.edu/helm/robo-reward-bench>.

1 INTRODUCTION

Despite recent algorithmic advances enabling efficient reinforcement learning (RL) training of robot control policies in the real world (Smith et al., 2022b; Luo et al., 2024; Mark et al., 2024; Ankile et al., 2025; Chen et al., 2025b; Wagenmaker et al., 2025), the broad application of RL to real-world robotics has been severely limited by the absence of **accurate and informative reward models**. RL-based methods critically require a **precise reward signal** to direct learning, yet existing methods for obtaining such rewards typically rely on either humans to label episodes by hand (Myers et al., 2023; Wagenmaker et al., 2025), or complex and brittle hand-crafted reward functions tuned by humans through extensive trial-and-error (Lee et al., 2020; Smith et al., 2022b; Luo et al., 2024; Chen et al., 2025b). While RL as an algorithmic paradigm holds the promise of enabling automated improvement of robot policies, the need for a human in the reward design process makes modern robotic RL labor-intensive, greatly limiting its application to general, real-world robotic policy improvement.

*Core contributors. Correspondence to tonyhlee@stanford.edu, ajwagen@berkeley.edu, pertsch@berkeley.edu.

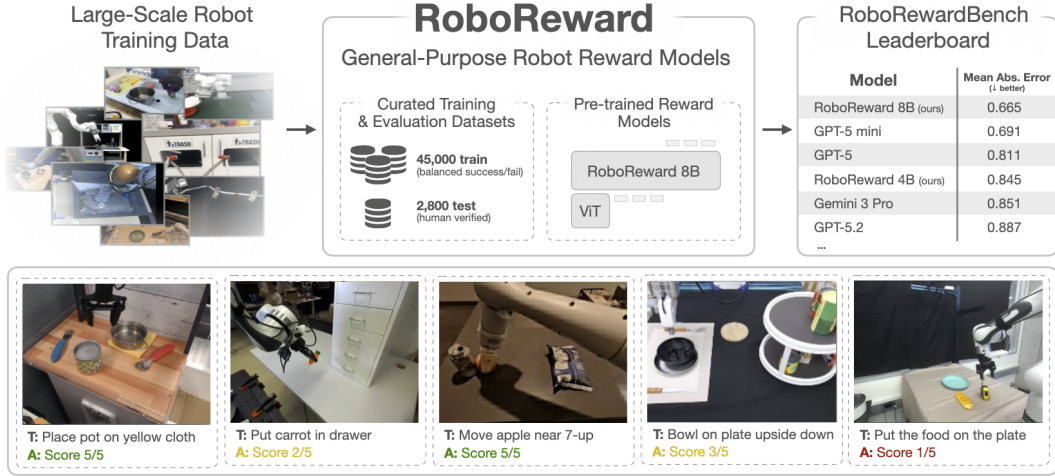


Figure 1: We introduce **RoboReward**, a dataset for training and evaluating general-purpose vision-language reward models for robotics. RoboReward consists of 2,800 real-robot episodes spanning diverse tasks and robots, with human-verified progress scores. In evaluations across 22 proprietary and open-source VLMs, we demonstrate that today’s models are lacking in their ability to provide accurate reward feedback for robots. We curate a training dataset of 45,000 scored robot episodes across diverse embodiments and train RoboReward 4B/8B, two general-purpose vision-language reward models for robotics that outperform frontier vision-language models. We open-source all models, training data, and our evaluation benchmark to advance the development of general-purpose reward models for robotics.

Motivated by these challenges, recent works have explored utilizing **vision-language models (VLMs)** trained on internet-scale data as automated reward models for robotics (Rocamonde et al., 2023; Venuto et al., 2024; Sontakke et al., 2024; Wang et al., 2024). In principle, a highly capable VLM that can reason about the physical world could replace hand-coded heuristics and expensive human supervision. However, existing methods often fall short of achieving this, due to **apparent shortcomings in current state-of-the-art VLMs** and **limited ability to provide sufficiently accurate rewards** in real-world robot deployments. While VLMs are pretrained on large datasets drawn from a diverse set of sources—endowing them with general vision-language abilities—it is not clear that these general abilities enable them, at present, to **robustly provide rewards at the level of precision and reliability required by RL training**.

In this work, we seek to develop a **dataset and benchmark for evaluating and improving VLM-based rewards for robotics**. In simple simulation experiments, we first identify that **coarse progress scores** are an effective reward type for reinforcement learning, and find that reward accuracy correlates with RL performance, motivating our benchmarking design choices in a controlled setting before scaling up to a diverse, real robot dataset. Unfortunately, existing large-scale robotics datasets (Open X-Embodiment Collaboration et al., 2023; Khazatsky et al., 2024) are **heavily skewed towards successful demonstration episodes**, which are poorly suited for training and evaluating reward functions for estimating both success and failure. We therefore develop a relabeling framework for **synthetically augmenting demonstration data**. Our framework counterfactually **relabels successful episodes with failed instructions and near-miss instructions for the same video**, holding the video of the episode fixed while varying the commanded task. We additionally generate *negative* and *partial-progress* examples by **temporally clipping successful videos to earlier endpoints**, yielding calibrated near-misses from the same underlying trajectory. We use these data augmentations to construct the **RoboReward dataset**: we augment success-heavy Open X-Embodiment (OXE) episodes (Open X-Embodiment Collaboration, 2023) with counterfactually relabeled and temporally clipped negatives, and we additionally include RoboArena (Atreya et al., 2025) as a complementary source of real-robot trajectories. Together, this yields an extensive training corpus and a human-validated evaluation dataset for reward modeling across diverse tasks and embodiments (see Figure 1).

A summary of our contributions is as follows:

1. **Negative examples data augmentation.** We augment success-heavy robot demonstration datasets with additional *wrong* and *near-miss* examples via *counterfactual relabeling*, and by *truncating successful rollouts* to create *partial-progress negatives*. In total, this pipeline yields 54,135 automatically generated examples across all splits, with the test set human-verified for benchmarking.
2. **Robot reward benchmarking and analysis.** We analyze supervision schemes for robotic rewards, comparing *binary success signals* to *discrete progress labels*. We also show that higher-quality robot reward models lead to stronger downstream RL policies. We then introduce **RoboRewardBench**, a comprehensive and standardized evaluation of VLMs as reward models on full robot rollouts, where we assess 22 prominent VLMs across 2,831 robot episodes spanning diverse tasks and 14 different types of embodiments with a mix of exocentric and egocentric camera perspectives.
3. **Resources.** We release the **RoboReward** training dataset, **RoboRewardBench** (the human-validated evaluation benchmark derived from the test split), and **trained reward-model checkpoints (RoboReward 4B and RoboReward 8B)** that outperform larger VLMs on assigning rewards for short-horizon tasks, along with an **evaluation suite** (including a leaderboard, prompts, raw generations, and results) to advance general-purpose reward modeling in robotics.

Our evaluation results indicate that current *general-purpose VLMs are not yet reliable reward models in all robotic control settings* and that the *RoboReward dataset can significantly improve reward accuracy over strong general-purpose VLM baselines*. Notably, our generalist 4B and 8B vision-language reward models rank **1** and **4** out of **22** on **RoboRewardBench**, outperforming substantially larger VLMs (including state-of-the-art proprietary models), and when used for real-world robotic RL, the **8B** model achieves higher task success than **Gemini Robotics-ER 1.5** and **substantially narrows the gap to human-provided rewards**.

2 RELATED WORK

Real-robot reinforcement learning. Autonomously learning and improving robotic control policies through reinforcement learning is a longstanding goal in the robotics community. Despite limited early success applying RL directly in the real world (Riedmiller et al., 2009; Levine et al., 2016; 2018), the majority of early work in this direction focused on learning in simulation and transferring the learned policy to the real world in deployment (Cutler et al., 2014; Rajeswaran et al., 2016; Tobin et al., 2017; Peng et al., 2018; Chebotar et al., 2019; Lee et al., 2020; Kumar et al., 2021). More recently, significant progress has been made applying RL to real-world locomotion (Smith et al., 2022b;a) and manipulation (Zhu et al., 2020; Luo et al., 2024; Mendonca et al., 2024; Luo et al., 2025b) settings. These works have primarily focused on learning from scratch or with a limited number of human demonstrations, yet with the advent of “generalist” robot policies (Octo Model Team et al., 2024; Kim et al., 2024; Black et al., 2024), significant attention has been devoted to developing RL algorithms that utilize such generalist policies as a starting point for learning, improving their behavior through RL in real-world deployment (Zhang et al., 2024; Mark et al., 2024; Nakamoto et al., 2024; Chen et al., 2025b; Hu et al., 2025; Ankile et al., 2025; Wagenmaker et al., 2025; Dong et al., 2025). All of these works, however, rely on either human reward supervision or hand-crafted reward functions in order to provide a signal for learning. This has greatly limited the application of RL to general robot learning settings, a challenge we aim to resolve in this work.

Learned reward models for robotics. To overcome the limitations of manually specified robot rewards, there is a long line of work for *learning robot reward functions*. Early works learned robot rewards from *human videos* (Sermanet et al., 2016; Shao et al., 2020; Chen et al., 2021) or *robot trajectories* (Ma et al., 2022; 2023; Yang et al., 2023; Sontakke et al., 2024). More recent works *leverage the expressivity and common sense of VLMs to derive rewards for control*. *Preference-based approaches query VLMs over image and trajectory comparisons or ratings to learn reward functions and train policies in simulation or the real world* (Wang et al., 2024; Venkataraman et al., 2024; Luu et al., 2025; Singh et al., 2025). A complementary direction *directly derives sparse or shaped rewards from individual robot videos* (Du et al., 2023; Rocamonde et al., 2023; Baumli et al., 2023; Yang et al., 2024a; Alakuijala et al., 2024; Yang et al., 2024b; Venuto et al., 2024). Ma et al. (2024) uses a VLM to perform *in-context value learning*. Zhang et al. (2025a) propose a reward relabeling scheme based on “rewinding” robot demonstrations, but their approach disregards

the content of the demonstration and is not evaluated using modern VLM models or diverse real robots. Other works target specific settings such as legged locomotion from videos (Zeng et al., 2024), text-to-video diffusion-based dense rewards (Chen et al., 2025a), autonomous driving with language-goal rewards (Huang et al., 2024), and real-to-sim iterative keypoint rewards (Patel et al., 2025). While these works demonstrate the promise of learned reward models for robotics, they typically focus on **a single reward model trained for an individual robot setup**. In contrast, our work presents, to our knowledge, the first comprehensive evaluation of 22 modern VLMs as **generalist reward functions across a wide range of robot tasks and embodiments**. Additionally, we provide an approach for counterfactual data relabeling that allows us to create large-scale training datasets for generalist reward functions and significantly improve over off-the-shelf models.

The most relevant work to our modeling setting is Tan et al. (2025), a concurrent work that introduces a *process* reward modeling approach for high-precision robotic manipulation. In contrast, our work targets general-purpose vision-language **end-of-episode reward prediction** across a broad range of real-robot tasks and embodiments, and contributes a comprehensive benchmark (RoboRewardBench) for evaluating many modern VLMs under a unified progress rubric. At the time of writing, their dataset and checkpoints have not been released, so we leave evaluating the Robo-Dopamine checkpoints with RoboRewardBench to future work.

The closest to our evaluation setting is the OpenGVL leaderboard (OpenGVL Team, 2025), which evaluates VLMs as temporal value estimators on expert videos using a Value-Order Correlation metric. OpenGVL defines only six tasks and reports results for 14 VLMs using only successful demonstration examples, however. In contrast, our work evaluates 22 VLMs, measuring their ability to predict rewards (rather than values) on a range of successful *and unsuccessful* trajectories, across diverse tasks and embodiments. We also release the prompts with videos and raw model predictions alongside our leaderboard for full transparency.

Non-robot reward models. With the recent success of RL approaches for post-training large language models (Shao et al., 2024; DeepSeek-AI et al., 2025), there has been a large number of works on training effective reward models for LLM post-training and RL (Lightman et al., 2023; Luo et al., 2025a). Additionally, a number of benchmarks have been proposed to evaluate these language reward models. For example, RewardBench (Lambert et al., 2024) and RewardBench 2 (Malik et al., 2025) test reward model accuracy, bias, and correlation with downstream LLM-RL performance. For multimodal settings, VLRewardBench (Li et al., 2024) and Multimodal RewardBench (Yasunaga et al., 2025) probe VLM reward models across perception, hallucination, reasoning, safety, and preference judgments. In contrast to these works, our focus is on reward functions for *robotic* tasks. As our evaluations show, the capabilities of current VLMs to adequately reward robot task performance lag far behind image or text domains, motivating our RoboReward benchmark.

3 THE ROLE OF REWARD IN REINFORCEMENT LEARNING

Reinforcement learning aims to find a **policy π** —a mapping from **states** to **actions**—that maximizes some **reward r** , typically a function of state and action. Formally, RL aims to find a policy π with maximum *expected* reward: $V^\pi := \mathbb{E}^\pi[\sum_{t \geq 0} \gamma^t r_t]$, where $\gamma \in [0, 1)$ denotes a discount factor, and r_t is the reward at step t . In robotics, we typically have some **actual objective we want to accomplish**, and the reward function must be specified such that **the policy learned by RL—the policy maximizing V^π —correctly achieves the desired objective**.

Our goal is to design a dataset for training and evaluating learned **generalist reward functions** in robotics. The first step is to choose a **reward function type** for our evaluation. For the purposes of this work, we restrict our investigation to **episodic rewards**, which assign a reward value to a full episode rather than each individual step, and have become the standard choice of reward in many applications of RL to robotics (Luo et al., 2024; Mark et al., 2024; Ankile et al., 2025; Chen et al., 2025b; Wagenmaker et al., 2025). Still, many design choices remain: episodic rewards can be **binary** or **multi-valued**, **discrete** or **continuous**. To guide the design of our **RoboReward** dataset, we first investigate **how the choice of reward formulation affects downstream RL performance in simulated RL tasks**. Concretely, we use the Robomimic benchmark (Mandlekar et al., 2021), a simulation suite that includes several robotic manipulation tasks. We seek to understand (a) what **type of reward** leads to RL training that quickly learns the desired tasks and (b) what is the correlation between the **accuracy of a learned reward model** and the **online RL performance**. In all experiments, we

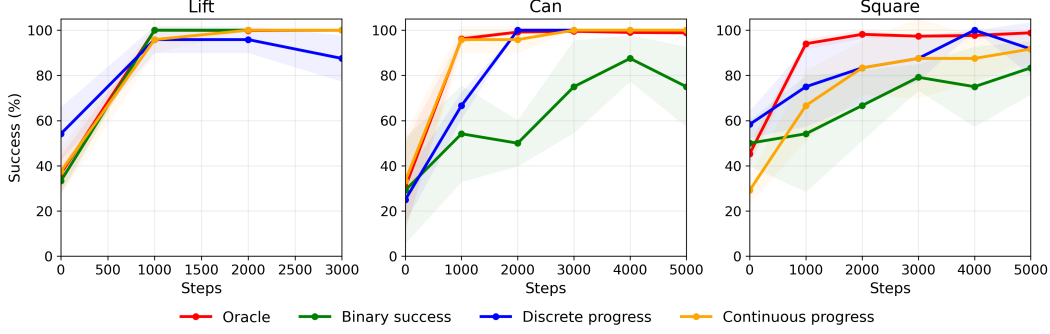


Figure 2: RL performance on three Robomimic tasks using learned reward functions with different reward formulations, averaged over 3 seeds (shaded regions show ± 1 standard deviation). Progress-based reward metrics lead to quicker convergence than a binary success metric. Both continuous and discrete progress rewards achieve comparably fast convergence. Thus, we choose *discrete progress* as the reward type for our benchmark, since it leads to quick convergence and is easier for humans to annotate consistently than continuous progress.

utilize DSRL (Wagenmaker et al., 2025)—a state-of-the-art RL fine-tuning algorithm—as our RL algorithm and apply it to finetune a diffusion policy pretrained on a dataset of task demonstrations included in Robomimic and ground truth rewards given by the simulation environment.

Which reward types lead to fast RL convergence? We first explore what type of reward leads to the most effective RL performance. In particular, as we are primarily interested in automated, learned reward models in this work, we seek to understand what type of *learned reward* leads to the most effective RL performance. We consider three different reward types:

1. **Binary success:** Reward is 1 if the robot episode succeeds, and 0 otherwise.
2. **Continuous progress:** Reward is a continuous value in $[0,1]$ corresponding to task progress given by the simulation environment.
3. **Discrete progress:** Similar to the continuous progress reward, but we discretize progress scores into 5 bins, and provide a reward in $\{1, \dots, 5\}$.

For each reward type, we programmatically label simulated Robomimic episodes using the simulator’s ground-truth reward signal, and finetune a Qwen2.5-VL model (Bai et al., 2025b) to predict these rewards from full-episode videos.

The RL finetuning results are given in Figure 2, where we plot the true success rate against the number of samples taken. We also plot the success rate of a policy finetuned with ground-truth (binary) rewards. We observe that the type of reward significantly impacts RL performance. In particular, while both learned progress rewards perform nearly as well as the ground truth rewards, the learned binary reward performs significantly worse. This suggests that *learning a progress reward for effective downstream RL performance is easier than learning a success reward* and, furthermore, that *whether this progress reward is discrete or continuous has minimal effect on RL performance*. Thus, we choose *discrete progress* as the reward formulation for RoboReward—we aim to learn a reward model that provides a progress score for a given task in $\{1, \dots, 5\}$ —since it is easier for humans to annotate consistently than fully continuous rewards.

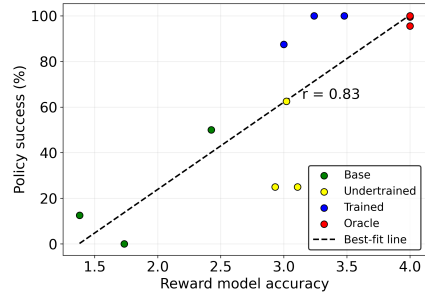


Figure 3: Strong positive correlation between reward accuracy and downstream RL performance, where the x-axis is maximum MAE minus the model MAE (larger is better; higher reward accuracy).

Do more accurate reward models lead to higher downstream RL performance? Next, we consider how the **accuracy of the learned reward model**—in particular, reward model accuracy evaluated on held-out sets of trajectories—affects **RL performance**. Our primary metric throughout the paper is **mean absolute error (MAE)** between predicted and ground-truth labels (lower is better), defined as $\text{MAE} := \frac{1}{N} \sum_{i=1}^N |\hat{r}_i - r_i|$, where $r_i \in \{1, \dots, 5\}$ is the ground-truth progress label for episode i and $\hat{r}_i \in \{1, \dots, 5\}$ is the model-predicted progress label. Intuitively, MAE measures the average distance between the predicted and true progress scores: for example, $\text{MAE} < 1$ means the model’s prediction is within one progress level of the ground truth on average (e.g., predicting 4 instead of 5 or 2 instead of 3).

Focusing on the discrete progress score reward from above, we measure reward accuracy on a held-out set of Robomimic validation episodes for multiple reward model checkpoints at different stages of convergence, as well as the off-the-shelf base model checkpoint. We then run RL to convergence with these reward models across all three Robomimic tasks. We show policy performance as a function of reward accuracy in Figure 3, where the x-axis plots the maximum possible MAE minus the model’s MAE (larger values mean higher accuracy). There is a clear correlation ($r = 0.83$): **more accurate rewards lead to better RL performance** across the board. These results suggest that evaluating the accuracy of a reward model on a held-out offline dataset is an effective signal for determining the performance of a downstream RL application that utilizes this reward model.

4 THE ROBOREWARD DATASET AND BENCHMARK

To train and evaluate highly capable general reward models for robotics, we need a **diverse dataset of real-world robot episodes** that span **successful and failed rollouts** and cover a **wide range of tasks and embodiments**. In recent years, multiple diverse real-robot datasets have been open-sourced (Open X-Embodiment Collaboration et al., 2023; Khazatsky et al., 2024; Walke et al., 2023; Fang et al., 2024; Mandlekar et al., 2018; Jiang et al., 2024; Bharadhwaj et al., 2024; Bu et al., 2025). However, most of these datasets are dominated by **successful demonstrations collected with expert policies or humans**. Although this is useful for training policies with behavioral cloning, it is suboptimal for training *reward models* that must discriminate fine-grained partial progress and failure. To address this imbalance, we introduce a **negative-examples data augmentation pipeline** that broadens the coverage of our reward model training corpus by synthesizing **partial success** and **failure** from success-heavy demonstrations. This pipeline combines (1) **counterfactual relabeling**: holding the video fixed while swapping in failed and near-miss instructions and (2) **episode clipping**: temporally truncating successful rollouts to create partial-progress outcomes. Our approach is loosely inspired by the popular hindsight experience relabeling technique (HER, Andrychowicz et al. (2017)), but instead of relabeling failed episodes as successes to increase the number of successful trials, we perform “inverse-HER” and relabel successes as failures to increase the number of unsuccessful trials and balance our training dataset. In this section, we describe the data sources we use to curate our RoboReward dataset, detail our relabeling procedure, and discuss the reward benchmark and the models we train on the RoboReward dataset.

4.1 DATA SOURCES

We aggregate real-robot videos from two primary data sources: the Open X-Embodiment dataset (OXE, Open X-Embodiment Collaboration et al. (2023)) and RoboArena (Atreya et al., 2025) evaluation data. Open X-Embodiment consists of approximately 1M real robot demonstrations, spanning 22 robot embodiments and numerous tasks, aggregated from a large number of individual academic and industry robot datasets. Each episode is paired with a natural-language instruction specifying the intended task (e.g., *place the mug in the sink*), alongside the rollout video. Since many datasets in OXE are highly repetitive (most demonstrations for an individual dataset may be collected in a single scene and task setup), we uniformly subsample up to 1200 episodes per dataset to reduce overfitting. Since all OXE episodes in our dataset are demonstrations, and therefore successful examples of the given task, we assign them the maximum reward score of 5.

RoboArena, on the other hand, is a diverse dataset of real-world robot policy evaluations across a broad range of scenes and tasks, using the DROID robot platform (Khazatsky et al., 2024). In RoboArena, human evaluators specify the natural-language tasks used for head-to-head policy evaluation and assign progress scores based on how well each policy’s rollout accomplishes the

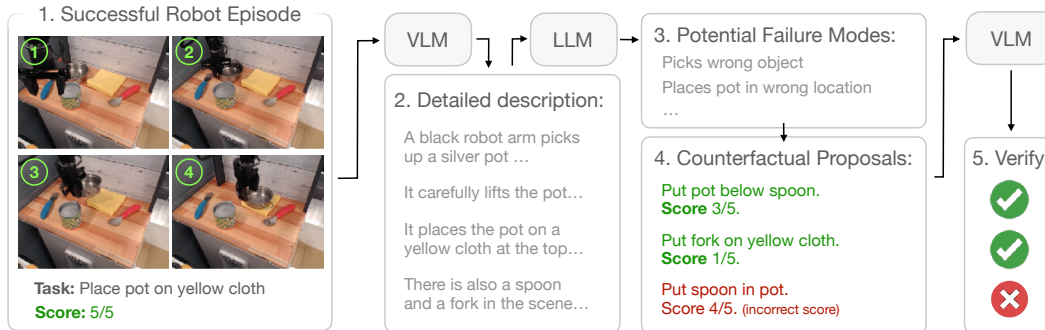


Figure 4: Overview of our counterfactual relabeling approach for generating *partial success* and *failure* task-video pairs for reward model training and evaluation. Given a successful robot episode, we use a VLM to describe it in detail, and then a sequence of LLM calls to propose alternative instructions for which the same video would result in only partial success or failure scores. A final VLM check verifies the quality of generated labels and rejects invalid labels.

intended instruction. Since there is comparatively less repetition in RoboArena and the dataset comprises a diverse mix of successful and natural failed policy rollouts, we opt to use the full dataset without subsampling. For each episode, we map the provided human progress score (originally in range $[0, 100]$) to discrete $1, \dots, 5$ rewards. For a complete list of all RoboReward data sources and their quantities, refer to Table 4.

4.2 DATA CLEANING AND NEGATIVE EXAMPLES DATA AUGMENTATION

We now describe our data cleaning and negative examples data augmentation pipeline. Full prompts for each stage are provided in Section A.2.

Prompt Rewriting. First, we normalize spelling and grammar without altering semantics, e.g., fixing spelling mistakes such as “*palce dishes in the dish rack*”. We apply a text-only rewrite transform that enforces semantic invariance: it preserves the meaning while improving the surface form. We use Qwen3 Instruct (4B) (Yang et al., 2025) for this transform (see Section A.2.1).

Negative Example Generation. Next, we address the imbalance of success vs. failure episodes in the data. We propose two complementary augmentations that **generate additional *wrong* training examples from a successful rollout video, without fabricating new videos**.

(i) Counterfactual relabeling. Given a successful robot rollout video, we synthesize **counterfactual task commands** for which the *same* video would only achieve **partial success**, or **no success at all** (see Figure 4). For example, given a video of a robot placing a pepper in a pot on the stove top, our pipeline may generate alternative commands such as `place pepper in the shelf` (partial progress, since the pepper was manipulated) or `clean the pot on the stove` (no success). This yields a richer reward training dataset with a more balanced distribution of successful and failed instruction-video pairs, and encourages reward models to attend closely to the task instruction.

(ii) Negative clipping. In addition to modifying the task text, we create **partial-progress outcomes** by **clipping successful videos to earlier endpoints**. Concretely, for each successful episode we generate **a small set of clipped videos that end at predetermined fractions of the rollout** (e.g., early, mid, and late cut points), while **keeping the original task instruction fixed**. These clipped rollouts often exhibit minimal progress or partial completion for the original task, providing additional negative and near-miss supervision that is grounded in the same trajectory.

Rubric and Validation. We assign **discrete end-state progress scores in $\{1, \dots, 5\}$** using a fixed rubric:

- *No success* (1): The final state shows no goal-relevant change for the task command.
- *Minimal progress* (2): The final state shows a small but insufficient change toward the goal.

- *Partial completion* (3): The final state shows good progress but violates major requirements or multiple requirements.
- *Near completion* (4): The final state is correct in region and intent but misses a single minor requirement.
- *Perfect completion* (5): The final state satisfies all the requirements.

After generating synthetic labels/examples, they run another VLM check to verify the generated sample is actually reasonable.

Basically train a smaller RoboReward 4B/8B model on the generated (task, video, reward) dataset.

Both counterfactual relabeling and negative clipping are filtered by a VLM-based validation step: we keep only *validated* examples whose task text is coherent and grounded in the video and whose provided score matches the rubric (see Section A.2.3 for exact prompts and additional details).

Our label generation is intentionally an *offline, high-cost pipeline*: it uses multiple stages (video analysis, rubric-grounded planning, constrained command generation, and rejection-based validation) and is therefore *too slow and operationally complex to run inside an RL loop at scale*. We instead use it as a *teacher* to produce weak but calibrated supervision that can be distilled into a single forward-pass, open-weight reward model suitable for repeated online evaluation. While the generated labels are not guaranteed to be perfect, two factors make them useful for training: (i) we aggressively filter with a strict rubric-based validation step and discard ambiguous or inconsistent examples, which reduces label noise, and (ii) the training mixture is not purely synthetic as RoboArena provides organically occurring successes and failures with human progress scores that help ground the rewards.

4.3 TRAINING AND EVALUATION OF GENERAL-PURPOSE ROBOT REWARD MODELS

We split the above corpus into a training and a test set. For each data source (OXE and RoboArena), we first partition by the *original task descriptions* provided by the dataset, and then assign episodes to train/validation/test such that task descriptions are disjoint across splits. This mitigates train–test contamination and provides a more stringent test of generalization to unseen tasks. With the negative example data augmentation, this results in a total training set of 45,072 episode-reward pairs, a validation set of 6,232, and a test set of 2,831 samples.

We use the training set to finetune **Qwen3-VL** (Bai et al., 2025a) at two scales (**4 billion** and **8 billion** parameters) to **predict the 5-level end-of-episode progress labels when given a task description and rollout video**. For both models, we **freeze the vision backbone** and **fine-tune the fusion and LLM layers**. We train for 3 epochs with cosine learning-rate decay, a warmup ratio of 0.05, weight decay of 0.05, and max gradient norm 1.0. We use an effective batch size of 32 via gradient accumulation, with learning rates of 3×10^{-6} (4B) and 5×10^{-6} (8B). For each scale, we select the best checkpoint that minimizes the mean absolute error (MAE) between the predicted and ground-truth 1–5 reward labels on a held-out validation set, producing trained vision-language reward models: **RoboReward 4B** and **RoboReward 8B**.

We designate the **test** split as our evaluation suite. We **further refine the test split by human-verifying every example** — the human annotator is asked to confirm that the end-state reward label is justified given the video of the rollout and task description (see Section A.3). We discard any example that does not pass human verification and then subsample from the remaining verified examples to form a clean evaluation set. We refer to this human-verified test split as **RoboReward-Bench**. Finally, our reported benchmark numbers are computed on **RoboRewardBench**, whose labels are **human-verified**, ensuring that evaluation remains trustworthy even if the training labels contain noise.

5 EXPERIMENTS

We next present experimental results in three parts. We begin by benchmarking off-the-shelf VLMs and our RoboReward models on RoboRewardBench (Section 5.1). We then evaluate whether these reward models enable effective real-world policy improvement via reinforcement learning (Section 5.2). Finally, we justify our data mixture and augmentation pipeline via data-mixture ablations that isolate the contributions of counterfactual relabeling and negative clipping to overall reward accuracy (Section 5.3).

Rank	Model	Overall (MAE) ↓	RoboArena ↓	OXE subsets ↓
1	RoboReward 8B (ours)	0.665	0.768	0.660
2	GPT-5 mini (2025-08-07)	0.691	0.862	0.683
3	GPT-5 (2025-08-07)	0.811	1.028	0.801
4	RoboReward 4B (ours)	0.845	0.806	0.847
5	Gemini 3 Pro (Preview)	0.851	1.234	0.833
		...		
7	Qwen3-VL Instruct (8B)	0.892	0.847	0.894
		...		
9	Gemini 2.5 Pro	0.902	0.936	0.900
		...		
11	Gemini Robotics-ER 1.5	0.906	1.002	0.902
		...		
13	Gemini 2.5 Flash	0.943	1.190	0.931
		...		
16	Qwen3-VL Instruct (4B)	1.032	0.929	1.036
		...		
21	Llama 4 Scout Instruct	1.485	1.830	1.469
22	Qwen2.5-VL Instruct (3B)	1.607	1.443	1.614

Table 1: RoboRewardBench results for a select subset of representative models. **Overall** reports the group-wise MAE (lower is better) over all RoboRewardBench subsets, **RoboArena** reports MAE on the RoboArena subset only, and **OXE subsets** reports the group-wise MAE averaged across the OXE-derived subsets. The fully expanded results and the complete ranking over all 22 models are in Table 10.

5.1 BENCHMARKING FRONTIER VLMS WITH ROBOREWARDBENCH

We evaluate 22 VLMS varying in size, model developers, and access (e.g., open weights vs. API) on RoboRewardBench, including our trained RoboReward VLMS (see Table 7 for the complete list). Each model is prompted with a task description and rollout video and must predict a discrete end-of-episode progress label in $\{1, 2, 3, 4, 5\}$. We measure performance using mean absolute error (MAE; lower is better), as defined in Section 3. Table 1 summarizes results for a select handful of representative models. For ranking the 22 models and full per-subset results, see Table 10, which reports the complete ranking and results for every RoboRewardBench subset.

Supervision with RoboReward yields capable general-purpose reward models. Our finetuned **RoboReward 8B** achieves the lowest overall MAE (0.665), outperforming all evaluated frontier VLMS, including GPT-5 mini (0.691) and Gemini family models (e.g., Gemini 2.5 Pro at 0.902, Gemini 2.5 Flash at 0.943). This trend also holds on the largest and most *organic* subset, **RoboArena** (1000 episodes). Both **RoboReward 8B** (0.768) and **RoboReward 4B** (0.806) outperform all evaluated frontier VLMS, improving over strong closed-access API baselines such as GPT-5 mini (0.862), GPT-5 (1.028), and Gemini Robotics-ER 1.5 (1.002), as well as Gemini 2.5 Pro (0.936) and Gemini 2.5 Flash (1.190).

Targeted supervision provides large gains at fixed architecture: **RoboReward (8B)** improves over **Qwen3-VL Instruct (8B)** by 0.227 MAE (0.892 to 0.665) and improves on **RoboArena** from 0.847 to 0.768, and **RoboReward (4B)** improves over **Qwen3-VL Instruct (4B)** by 0.187 MAE (1.032 to 0.845) and improves on **RoboArena** from 0.929 to 0.806. Together, these results suggest that *high-quality, diverse reward supervision* can substantially improve reward prediction accuracy.

RoboRewardBench reveals large, non-uniform generalization gaps. The per-subset columns in Table 10 show pronounced performance swings across embodiments, scenes, and viewpoints. For instance, **GPT-5 (2025-08-07)** is among the top models on RoboRewardBench overall (MAE = 0.811), yet its accuracy varies widely across subsets: it is relatively strong on *Austin Sirius* (0.310)

and *Austin VIOLA* (0.283), but substantially worse on *Berkeley RPT* (1.174), *Tokyo PR2 Tabletop Manipulation* (1.366), and *RoboArena* (1.028). A similar non-uniformity appears for **Gemini 3 Pro (Preview)**, Google DeepMind’s flagship model. Gemini 3 Pro is the best-performing model on *Berkeley RPT* (0.395) and performs strongly on *UCSD Pick Place* (0.37), yet it degrades sharply on other subsets such as *Berkeley MVP* (1.394) and *KAIST Nonprehensile Objects* (1.491).

This generalization gap is a challenge even for models trained on robotics data. **Gemini Robotics-Embodied Reasoning 1.5** (Gemini Robotics-ER 1.5, Google DeepMind (2025f)), a frontier model trained on robotics data to excel in progress estimation and embodied reasoning, attains an overall MAE of 0.906 on RoboRewardBench, ranking 11th out of 22 models in Table 1. At the same time, despite **RoboReward (8B)** ranked as the best model overall, it is not uniformly best across subsets. For example, it is comparatively weak on *UTokyo xArm Bimanual* (absolute error 1.394), *Stanford HYDRA* (0.890, rank 8/22), while the best model (**Gemini 2.5 Pro**) achieves an MAE of 0.495. This echoes broader findings that real-world physical reasoning remains challenging even for models trained specifically on robot data and frontier VLMs (Pătrăucean et al., 2023; Lee et al., 2024; Zhang et al., 2025b).

5.2 TRAINING REAL-ROBOT POLICIES WITH VLM REWARD MODELS

Finally, we aim to demonstrate that RoboReward provides a sufficiently accurate reward signal to enable real-world robotic RL.

Setting. For the real-world experiments, we utilize the WidowX 250 6-DoF robot arm. As our RL algorithm, we run DSRL to finetune a multi-task diffusion policy trained on the BridgeData V2 dataset (Walke et al., 2023). For a reward signal, we use a sparse end-of-episode reward, comparing the following three settings: (1) oracle human reward: a human labeler gives a positive reward of +1 on success and the reward is otherwise 0, (2) RoboReward 8B: outputs a 1-5 progress score at the end of each episode, and (3) Gemini Robotics-ER 1.5: outputs a progress score 1-5, similar to RoboReward. We note that Gemini Robotics-ER 1.5 is a frontier model that was specifically trained for embodied reasoning and robotics. **Both VLM reward models are not further fine-tuned and are prompted zero-shot.** We consider two real-world tasks for the WidowX robot to perform in two different settings, neither of which was seen during training. The first task is to pick up the brown monkey and move it on top of the yellow towel (Pick-and-place monkey). The second task is to pull the drawer out (Open drawer). See Figure 5 for an illustration of each task. For both tasks and across the three reward settings, we train for 6000 steps, where an episode is a maximum of 70 steps (see Section B.4 for full hyperparameter and training details).



Figure 5: Real robot tasks: *Pick up the brown monkey and move it on top of the yellow towel* (Pick-and-place monkey, left) and *Pull the drawer out* (Open drawer, right). We run real-world RL improvement on each of these tasks, using RoboReward as a reward.

We consider two real-world tasks for the WidowX robot to perform in two different settings, neither of which was seen during training. The first task is to pick up the brown monkey and move it on top of the yellow towel (Pick-and-place monkey). The second task is to pull the drawer out (Open drawer). See Figure 5 for an illustration of each task. For both tasks and across the three reward settings, we train for 6000 steps, where an episode is a maximum of 70 steps (see Section B.4 for full hyperparameter and training details).

Table 2: Performance (success rate over 20 trials) of running RL with various reward models compared to the base policy. Values in parentheses show the change compared to the base policy before training with RL. **RoboReward 8B** closes much of the gap to **human rewards** and substantially outperforms **Gemini Robotics-ER 1.5** as a reward model.

Rewards	Pick-and-place monkey	Open drawer
Base policy	5%	10%
Human	75% (+70)	90% (+80)
RoboReward 8B	50% (+45)	80% (+70)
Gemini Robotics-ER 1.5	10% (+5)	45% (+35)

Results. The results, obtained from 20 trials per task across the different settings, are summarized in Table 2. We see that RoboReward 8B enables effective real-world RL improvement, and that running RL with this reward improves the performance of the base policy on both tasks (from 5% to 50% on pick-and-place monkey, and 10% to 80% on open drawer). While the performance of RL improvement with RoboReward does not yet match the performance achieved with the oracle human reward (which reaches 75% and 90%, respectively, on the two tasks), it closes much of the gap, especially on `Open drawer` (80% vs. 90%). Furthermore, RoboReward significantly outperforms Gemini Robotics-ER 1.5, which fails to yield any improvement on `Pick-and-place monkey`, and only minimal improvement on `Open drawer`.

We also note that the ordering of models by the real-world RL performance they induce is consistent with their performance on RoboRewardBench. In particular, Gemini Robotics-ER 1.5 has a higher overall mean absolute error on RoboRewardBench (0.906), and it underperforms RoboReward 8B as a reward model, yielding substantially weaker downstream RL gains, while RoboReward 8B has a lower mean absolute error on RoboRewardBench (0.665) and produces much greater improvements over the base policy on both tasks. These results align with our previous findings from our simulation experiments that better reward quality leads to improved downstream RL performance, and they further stress the importance of training high-quality reward models for robotics reinforcement learning. We further discuss qualitative failure modes of VLM reward models in Section B.3, which help explain the remaining gap between human-assigned rewards and VLM-predicted rewards.

5.3 DATA MIXTURE ABLATIONS

We evaluate how different sources of negative supervision affect the accuracy of reward models on RoboRewardBench. Our full training set includes (1) successful OXE demonstrations augmented with counterfactual relabeling and temporally clipped versions of the same videos to create visually grounded negatives and near-misses, and (2) RoboArena rollouts on the DROID platform that contain organic successes and failures. Using the 4B model, we run two data-mixture ablations: (1) train on RoboArena plus the original (success-only) OXE demonstrations with no negative examples augmentation, and (2) remove the clipped-video negatives while keeping the counterfactual relabeling. Table 3 summarizes the results. From these ablations, we highlight two main takeaways.

Data mixture	Overall (MAE) ↓	RoboArena ↓
Full	0.845	0.806
Full – Neg. Aug.	1.450	0.797
Full – Neg. clipping	1.075	0.813

Table 3: Results from data mixture ablations on RoboRewardBench (lower is better). **Overall** reports MAE on the full RoboRewardBench (including RoboArena), while **RoboArena** reports MAE on the RoboArena subset only. Training with RoboArena and OXE success examples only (without negative examples augmentation) matches in-domain errors but fails to generalize, while training on the **full RoboReward dataset** (counterfactual labels, clipped examples, and RoboArena) achieves matching performance on RoboArena and substantially improves overall performance.

RoboArena failures alone are insufficient for broad generalization. Training only on RoboArena and the original OXE examples achieves nearly the same error on the RoboArena subset of RoboRewardBench as the training on the full data mixture (MAE 0.797 vs. 0.806). However, just training on RoboArena and OXE performs dramatically worse on the overall benchmark (1.45 vs. 0.845). This gap suggests that while RoboArena provides high-quality organic failures, it is limited in coverage (single platform) and does not expose the model to the broader distribution of instruction-conditioned failures present in RoboRewardBench. In contrast, counterfactual relabeling and clipping introduce diverse negative outcomes across many scenes and embodiments, which is necessary for a general-purpose reward model.

Clipping episodes improves robustness without harming organic performance. Removing clipped negative examples degrades overall performance (1.075 vs 0.845), while having minimal impact on RoboArena (0.813 vs 0.806), suggesting clipped examples primarily broaden coverage of failure modes and are critical to achieving effective performance across RoboRewardBench.

Therefore, high-quality organic data from RoboArena is valuable, but to achieve strong performance across RoboRewardBench, the reward model needs broad and instruction-sensitive negative coverage, which is provided by counterfactual relabeling and reinforced by negative clipping.

6 DISCUSSION

In this work, we introduced the **RoboReward** training dataset, **RoboRewardBench**, a human-verified evaluation suite for benchmarking generalist vision-language reward models on real-robot rollouts, and two finetuned reward VLMs (**RoboReward 4B/8B**). Across 22 frontier and open-weight models, RoboReward 8B achieves the best overall accuracy on RoboRewardBench, demonstrating that targeted reward supervision can outperform substantially larger general-purpose VLMs. Further, we show in both simulation and real-robot settings that improvements in offline reward accuracy translate into improved downstream RL performance, making benchmark progress a meaningful proxy for real-world usefulness.

More broadly, our results suggest that reward modeling is a key bottleneck for real-world RL: current frontier VLMs still exhibit large and non-uniform generalization gaps across embodiments and scenes, and even small reward mistakes can meaningfully impact policy improvement. By releasing a standardized benchmark with human-verified labels and an open evaluation suite, we aim to make these limitations measurable and to accelerate progress on reliable, general-purpose reward models for robotics. An important direction for future work is extending reward modeling to longer-horizon, multi-stage tasks, where credit assignment and progress estimation become more challenging.

ACKNOWLEDGMENTS

We thank Yifan Mai for maintaining the HELM codebase, which was used for benchmarking and populating the leaderboard. This research was partly supported by ONR N00014-22-1-2621, ONR N00014-25-1-2060, and the Toyota Research Institute (TRI).

REFERENCES

- Minttu Alakuijala, Reginald McLean, Isaac Woungang, Nariman Farsad, Samuel Kaski, Pekka Marttinen, and Kai Yuan. Video-language critic: Transferable reward functions for language-conditioned robotics. *arXiv preprint arXiv:2405.19988*, 2024. URL <https://arxiv.org/abs/2405.19988>.
- Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *NeurIPS*, 2017.
- Lars Ankile, Anthony Simeonov, Idan Shenfeld, Marcel Torne, and Pulkit Agrawal. From imitation to refinement-residual rl for precise assembly. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 01–08. IEEE, 2025.
- Pranav Atreya, Karl Pertsch, Tony Lee, Moo Jin Kim, Arhan Jain, Artur Kuramshin, Clemens Eppner, Cyrus Neary, Edward Hu, Fabio Ramos, Jonathan Tremblay, Kanav Arora, Kirsty Ellis, Luca Maccesanu, Matthew Leonard, Meedeum Cho, Ozgur Aslan, Shivin Dass, Jie Wang, Xingfang Yuan, Xuning Yang, Abhishek Gupta, Dinesh Jayaraman, Glen Berseth, Kostas Daniilidis, Roberto Martín-Martín, Youngwoon Lee, Percy Liang, Chelsea Finn, and Sergey Levine. Roboarena: Distributed real-world evaluation of generalist robot policies. *arXiv preprint arXiv:2506.18123*, 2025. URL <https://arxiv.org/abs/2506.18123>.
- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun

- Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-vl technical report, 2025a. URL <https://arxiv.org/abs/2511.21631>.
- Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-vl technical report, 2025b. URL <https://arxiv.org/abs/2502.13923>.
- Kate Baumli, Satinder Baveja, Feryal Behbahani, Harris Chan, Gheorghe Comanici, Sebastian Flennerhag, Maxime Gazeau, Kristian Holsheimer, Dan Horgan, Michael Laskin, et al. Vision-language models as a source of rewards. *arXiv preprint arXiv:2312.09187*, 2023.
- Suneel Belkale, Yuchen Cui, and Dorsa Sadigh. Hydra: Hybrid robot actions for imitation learning. In *Robotics: Science and Systems (RSS)*, 2023.
- Homanga Bharadhwaj, Jay Vakil, Mohit Sharma, Abhinav Gupta, Shubham Tulsiani, and Vikash Kumar. Roboagent: Generalization and efficiency in robot manipulation via semantic augmentations and action chunking. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4788–4795. IEEE, 2024.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. *pi_0*: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Shenyuan Gao, Xindong He, Xuan Hu, Xu Huang, et al. Agibot world colosseum: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025.
- Yevgen Chebotar, Ankur Handa, Viktor Makoviychuk, Miles Macklin, Jan Issac, Nathan Ratliff, and Dieter Fox. Closing the sim-to-real loop: Adapting simulation randomization with real world experience. In *ICRA*, 2019.
- Annie S Chen, Suraj Nair, and Chelsea Finn. Learning generalizable robotic reward functions from “in-the-wild” human videos. *RSS*, 2021.
- Lawrence Yunliang Chen, Simeon Adebola, and Ken Goldberg. Berkeley ur5 demonstration dataset. <https://sites.google.com/view/berkeley-ur5/home>.
- Yuhui Chen, Haoran Li, Zhennan Jiang, Haowei Wen, and Dongbin Zhao. Tevir: Text-to-video reward with diffusion models for efficient reinforcement learning. *arXiv preprint arXiv:2505.19769*, 2025a. URL <https://arxiv.org/abs/2505.19769>.
- Yuhui Chen, Shuai Tian, Shugao Liu, Yingting Zhou, Haoran Li, and Dongbin Zhao. Conrft: A reinforced fine-tuning method for vla models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025b.
- Mark Cutler, Thomas J Walsh, and Jonathan P How. Reinforcement learning with multi-fidelity simulators. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3888–3895. IEEE, 2014.
- Sudeep Dasari, Oier Mees, Sebastian Zhao, Mohan Kumar Srirama, and Sergey Levine. The ingredients for robotic diffusion transformers. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 15617–15625. IEEE, 2025.
- Shivin Dass, Jullian Yapeter, Jesse Zhang, Jiahui Zhang, Karl Pertsch, Stefanos Nikolaidis, and Joseph J. Lim. Clvr jaco play dataset, 2023. URL https://github.com/clvr-ai/clvr_jaco_play_dataset.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaoshan Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.

Perry Dong, Suvir Mirchandani, Dorsa Sadigh, and Chelsea Finn. What matters for batch online reinforcement learning in robotics? *arXiv preprint arXiv:2505.08078*, 2025.

Yuqing Du, Ksenia Konyushkova, Misha Denil, Akhil Raju, Jessica Landon, Felix Hill, Nando de Freitas, and Serkan Cabi. Vision-language models as success detectors. *arXiv preprint arXiv:2303.07280*, 2023.

Hao-Shu Fang, Hongjie Fang, Zhenyu Tang, Jirong Liu, Chenxi Wang, Junbo Wang, Haoyi Zhu, and Cewu Lu. Rh20t: A comprehensive robotic dataset for learning diverse skills in one-shot. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 653–660. IEEE, 2024.

Yunhai Feng, Nicklas Hansen, Ziyan Xiong, Chandramouli Rajagopalan, and Xiaolong Wang. Fine-tuning offline world models in the real world. *arXiv preprint arXiv:2312.00000*, 2023.

Google DeepMind. Gemini 2.5 Flash-Lite Model Card. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-2-5-Flash-Lite-Model-Card.pdf>, 2025a. Accessed: 2025-12-23.

Google DeepMind. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. https://storage.googleapis.com/deepmind-media/gemini/gemini_v2_5_report.pdf, 2025b. Accessed: 2025-12-23.

Google DeepMind. Gemini 3 Flash Model Card. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Flash-Model-Card.pdf>, 2025c. Accessed: 2025-12-23.

Google DeepMind. Gemini 3 Pro Model Card. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf>, 2025d. Accessed: 2025-12-23.

- Google DeepMind. Gemini Robotics 1.5: Pushing the Frontier of Generalist Robotics. <https://storage.googleapis.com/deepmind-media/gemini-robotics/Gemini-Robotics-1-5-Tech-Report.pdf>, 2025e. Accessed: 2025-12-23.
- Google DeepMind. Gemini Robotics ER. <https://deepmind.google/models/gemini-robotics/gemini-robotics-er/>, 2025f. Accessed: 2025-12-23.
- Siddhant Haldar, Vaibhav Mathur, Denis Yarats, and Lerrel Pinto. Watch and match: Supercharging imitation with regularized optimal transport. In *Conference on Robot Learning (PMLR)*, 2023.
- Jiaheng Hu, Rose Hendrix, Ali Farhadi, Aniruddha Kembhavi, Roberto Martín-Martín, Peter Stone, Kuo-Hao Zeng, and Kiana Ehsani. Flare: Achieving masterful and adaptive robot policies with large-scale reinforcement learning fine-tuning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3617–3624. IEEE, 2025.
- Zilin Huang, Zihao Sheng, Yansong Qu, Junwei You, and Sikai Chen. Vlm-rl: A unified vision–language model and reinforcement learning framework for safe autonomous driving. *arXiv preprint arXiv:2412.15544*, 2024. URL <https://arxiv.org/abs/2412.15544>.
- Zhenyu Jiang, Yuqi Xie, Kevin Lin, Zhenjia Xu, Weikang Wan, Ajay Mandlekar, Linxi Fan, and Yuke Zhu. Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning. *arXiv preprint arXiv:2410.24185*, 2024.
- Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, Peter David Fagan, Joey Hejna, Masha Itkina, Marion Lepert, Yecheng Jason Ma, Patrick Tree Miller, Jimmy Wu, Suneel Belkhale, Shivin Dass, Huy Ha, Arhan Jain, Abraham Lee, Young-woon Lee, Marius Memmel, Sungjae Park, Ilija Radosavovic, Kaiyuan Wang, Albert Zhan, Kevin Black, Cheng Chi, Kyle Beltran Hatch, Shan Lin, Jingpei Lu, Jean Mercat, Abdul Rehman, Pannag R Sanketi, Archit Sharma, Cody Simpson, Quan Vuong, Homer Rich Walke, Blake Wulfe, Ted Xiao, Jonathan Heewon Yang, Arefeh Yavary, Tony Z. Zhao, Christopher Agia, Rohan Bajjal, Mateo Guaman Castro, Daphne Chen, Qiuyu Chen, Trinity Chung, Jaimyn Drake, Ethan Paul Foster, Jensen Gao, David Antonio Herrera, Minho Heo, Kyle Hsu, Jiaheng Hu, Donovan Jackson, Charlotte Le, Yunshuang Li, Kevin Lin, Roy Lin, Zehan Ma, Abhiram Maddukuri, Suvir Mirchandani, Daniel Morton, Tony Nguyen, Abigail O’Neill, Rosario Scalise, Derick Seale, Victor Son, Stephen Tian, Emi Tran, Andrew E. Wang, Yilin Wu, Annie Xie, Jingyun Yang, Patrick Yin, Yunchu Zhang, Osbert Bastani, Glen Berseth, Jeannette Bohg, Ken Goldberg, Abhinav Gupta, Abhishek Gupta, Dinesh Jayaraman, Joseph J Lim, Jitendra Malik, Roberto Martín-Martín, Subramanian Ramamoorthy, Dorsa Sadigh, Shuran Song, Jiajun Wu, Michael C. Yip, Yuke Zhu, Thomas Kollar, Sergey Levine, and Chelsea Finn. Droid: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. Rma: Rapid motor adaptation for legged robots. *arXiv preprint arXiv:2107.04034*, 2021.
- Nathan Lambert, Valentina Pyatkin, Jacob Morrison, L. J. Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. Rewardbench: Evaluating reward models for language modeling. *arXiv preprint arXiv:2403.13787*, 2024. URL <https://arxiv.org/abs/2403.13787>.
- Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5(47):eabc5986, 2020.
- Tony Lee, Haoqin Tu, Chi Heem Wong, Wenhao Zheng, Yiyang Zhou, Yifan Mai, Joselin Somerville Roberts, Michihiro Yasunaga, Huaxiu Yao, Cihang Xie, and Percy Liang. Vhelm: A holistic evaluation of vision language models, 2024. URL <https://arxiv.org/abs/2410.07112>.

- Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *The Journal of Machine Learning Research*, 17(1):1334–1373, 2016.
- Sergey Levine, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International journal of robotics research*, 37(4-5):421–436, 2018.
- Lei Li, Yuancheng Wei, Zihui Xie, Xuqing Yang, Yifan Song, Peiyi Wang, Chenxin An, Tianyu Liu, Sujian Li, Bill Yuchen Lin, Lingpeng Kong, and Qi Liu. Vrewardbench: A challenging benchmark for vision–language generative reward models. *arXiv preprint arXiv:2411.17451*, 2024. URL <https://arxiv.org/abs/2411.17451>.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- Huihan Liu, Soroush Nasiriany, Lance Zhang, Zhiyao Bao, and Yuke Zhu. Robot learning on the job: Human-in-the-loop autonomy and learning during deployment. In *Robotics: Science and Systems (RSS)*, 2023.
- Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, Yansong Tang, and Dongmei Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct, 2025a. URL <https://arxiv.org/abs/2308.09583>.
- Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. Serl: A software suite for sample-efficient robotic reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 16961–16969. IEEE, 2024.
- Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105):eads5033, 2025b.
- Tung M. Luu, Younghwan Lee, Donghoon Lee, Sunho Kim, Min Jun Kim, and Chang D. Yoo. Erl-vm: Enhancing rating-based reinforcement learning to effectively leverage feedback from large vision–language models. *arXiv preprint arXiv:2506.12822*, 2025. URL <https://arxiv.org/abs/2506.12822>.
- Corey Lynch, Ayzaan Wahid, Jonathan Tompson, Tianli Ding, James Betker, Robert Baruch, Travis Armstrong, and Pete Florence. Interactive language: Talking to robots in real time. *IEEE Robotics and Automation Letters*, 2023.
- Yecheng Jason Ma, Shagun Sodhani, Dinesh Jayaraman, Osbert Bastani, Vikash Kumar, and Amy Zhang. Vip: Towards universal visual reward and representation via value-implicit pre-training. In *The Eleventh International Conference on Learning Representations*, 2022.
- Yecheng Jason Ma, Vikash Kumar, Amy Zhang, Osbert Bastani, and Dinesh Jayaraman. Liv: Language-image representations and rewards for robotic control. In *International Conference on Machine Learning*, pp. 23301–23320. PMLR, 2023.
- Yecheng Jason Ma, Joey Hejna, Ayzaan Wahid, Chuyuan Fu, Dhruv Shah, Jacky Liang, Zhuo Xu, Sean Kirmani, Peng Xu, Danny Driess, Ted Xiao, Jonathan Tompson, Osbert Bastani, Dinesh Jayaraman, Wenhao Yu, Tingnan Zhang, Dorsa Sadigh, and Fei Xia. Vision language models are in-context value learners. *arXiv preprint arXiv:2411.04549*, 2024. URL <https://arxiv.org/abs/2411.04549>.
- Saumya Malik, Valentina Pyatkin, Sander Land, Jacob Morrison, Noah A. Smith, Hannaneh Hajishirzi, and Nathan Lambert. Rewardbench 2: Advancing reward model evaluation. *arXiv preprint arXiv:2506.01937*, 2025. URL <https://arxiv.org/abs/2506.01937>.
- Ajay Mandlekar, Yuke Zhu, Animesh Garg, Jonathan Bozher, Max Spero, Albert Tong, Julian Gao, John Emmons, Anchit Gupta, Emre Orbay, et al. Roboturk: A crowdsourcing platform for robotic skill learning through imitation. In *Conference on Robot Learning*, pp. 879–893. PMLR, 2018.

- Ajay Mandlekar, Jonathan Booher, Max Spero, Albert Tung, Anchit Gupta, Yuke Zhu, Animesh Garg, Silvio Savarese, and Li Fei-Fei. Scaling robot supervision to hundreds of hours with roboturk: Robotic manipulation dataset through human reasoning and dexterity. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1048–1055, 2019.
- Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *arXiv preprint arXiv:2108.03298*, 2021.
- Max Sobol Mark, Tian Gao, Georgia Gabriela Sampaio, Mohan Kumar Srirama, Archit Sharma, Chelsea Finn, and Aviral Kumar. Policy agnostic rl: Offline rl and online rl fine-tuning of any class and backbone. *arXiv preprint arXiv:2412.06685*, 2024.
- Tatsuya Matsushima, Hiroki Furuta, Yusuke Iwasawa, and Yutaka Matsuo. Weblab xarm datasets. <https://github.com/weblab-xarm>, 2023.
- Oier Mees, Jessica Borja-Diaz, and Wolfram Burgard. Grounding language with visual affordances over unstructured data. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- Russell Mendonca, Emmanuel Panov, Bernadette Bucher, Jiuguang Wang, and Deepak Pathak. Continuously improving mobile manipulation with autonomous real-world rl. *arXiv preprint arXiv:2409.20568*, 2024.
- Meta. Llama 4: Model Cards and Prompt Formats. <https://www.llama.com/docs/model-cards-and-prompt-formats/llama4/>, 2025. Accessed: 2025-12-23.
- Vivek Myers, Erdem Bıyık, and Dorsa Sadigh. Active reward learning from online preferences. *arXiv preprint arXiv:2302.13507*, 2023.
- Mitsuhiko Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. *arXiv preprint arXiv:2410.13816*, 2024.
- Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Charles Xu, Jianlan Luo, Tobias Kreiman, You Liang Tan, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.
- Jihoon Oh, Naoaki Kanazawa, and Kento Kawaharazuka. X-embodiment u-tokyo pr2 datasets. https://github.com/ojh6404/rlds_dataset_builder, 2023.
- Open X-Embodiment Collaboration. Open x-embodiment: Robotic learning datasets and rt-x models. *arXiv preprint arXiv:2310.08864*, 2023. doi: 10.48550/arXiv.2310.08864. URL <https://arxiv.org/abs/2310.08864>.
- Open X-Embodiment Collaboration, Abhishek Padalkar, Acorn Pooley, Ajinkya Jain, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anikait Singh, Anthony Brohan, Antonin Raffin, Ayzaan Wahid, Ben Burgess-Limerick, Beomjoon Kim, Bernhard Schölkopf, Brian Ichter, Cewu Lu, Charles Xu, Chelsea Finn, Chenfeng Xu, Cheng Chi, Chenguang Huang, Christine Chan, Chuer Pan, Chuyuan Fu, Coline Devin, Danny Driess, Deepak Pathak, Dhruv Shah, Dieter Büchler, Dmitry Kalashnikov, Dorsa Sadigh, Edward Johns, Federico Ceola, Fei Xia, Freck Stulp, Gaoyue Zhou, Gaurav S. Sukhatme, Gautam Salhotra, Ge Yan, Giulio Schiavi, Hao Su, Hao-Shu Fang, Haochen Shi, Heni Ben Amor, Henrik I Christensen, Hiroki Furuta, Homer Walke, Hongjie Fang, Igor Mordatch, Ilija Radosavovic, Isabel Leal, Jacky Liang, Jaehyung Kim, Jan Schneider, Jasmine Hsu, Jeannette Bohg, Jeffrey Bingham, Jiajun Wu, Jialin Wu, Jianlan Luo, Jiayuan Gu, Jie Tan, Jihoon Oh, Jitendra Malik, Jonathan Tompson, Jonathan Yang, Joseph J. Lim, João Silvério, Junhyek Han, Kanishka Rao, Karl Pertsch, Karol Hausman, Keegan Go, Keerthana Gopalakrishnan, Ken Goldberg, Kendra Byrne, Kenneth Oslund, Kento Kawaharazuka, Kevin Zhang, Keyvan Majd, Krishan Rana, Krishnan Srinivasan, Lawrence Yunliang Chen, Lerrel Pinto, Liam Tan, Lionel Ott, Lisa Lee, Masayoshi Tomizuka, Maximilian Du, Michael Ahn, Mingtong Zhang, Mingyu Ding, Mohan Kumar Srirama, Mohit Sharma, Moo Jin

- Kim, Naoaki Kanazawa, Nicklas Hansen, Nicolas Heess, Nikhil J Joshi, Niko Suenderhauf, Norman Di Palo, Nur Muhammad Mahi Shafiullah, Oier Mees, Oliver Kroemer, Pannag R Sanketi, Paul Wohlhart, Peng Xu, Pierre Sermanet, Priya Sundaresan, Quan Vuong, Rafael Rafailov, Ran Tian, Ria Doshi, Roberto Martín-Martín, Russell Mendonca, Rutav Shah, Ryan Hoque, Ryan Julian, Samuel Bustamante, Sean Kirmani, Sergey Levine, Sherry Moore, Shikhar Bahl, Shivin Dass, Shuran Song, Sichun Xu, Siddhant Halder, Simeon Adebola, Simon Guist, Soroush Nasiriany, Stefan Schaal, Stefan Welker, Stephen Tian, Sudeep Dasari, Suneel Belkhale, Takayuki Osa, Tatsuya Harada, Tatsuya Matsushima, Ted Xiao, Tianhe Yu, Tianli Ding, Todor Davchev, Tony Z. Zhao, Travis Armstrong, Trevor Darrell, Vidhi Jain, Vincent Vanhoucke, Wei Zhan, Wenxuan Zhou, Wolfram Burgard, Xi Chen, Xiaolong Wang, Xinghao Zhu, Xuanlin Li, Yao Lu, Yevgen Chebotar, Yifan Zhou, Yifeng Zhu, Ying Xu, Yixuan Wang, Yonatan Bisk, Yoonyoung Cho, Youngwoon Lee, Yuchen Cui, Yueh hua Wu, Yujin Tang, Yuke Zhu, Yunzhu Li, Yusuke Iwasawa, Yutaka Matsuo, Zhuo Xu, and Zichen Jeff Cui. Open X-Embodiment: Robotic learning datasets and RT-X models. <https://arxiv.org/abs/2310.08864>, 2023.
- OpenAI. GPT-5.1 Instant and GPT-5.1 Thinking System Card. https://cdn.openai.com/pdf/4173ec8d-1229-47db-96de-06d87147e07e/5_1_system_card.pdf, 2025a. Accessed: 2025-12-23.
- OpenAI. Update to GPT-5 System Card: GPT-5.2. https://cdn.openai.com/pdf/3a4153c8-c748-4b71-8e31-aecbde944f8d/oai_5_2_system-card.pdf, 2025b. Accessed: 2025-12-23.
- OpenAI. GPT-5 System Card. <https://cdn.openai.com/gpt-5-system-card.pdf>, 2025c. Accessed: 2025-12-23.
- OpenGVL Team. Opengvl: Task completion leaderboard for evaluating vlms as temporal value estimators. <https://huggingface.co/spaces/OpenGVL/OpenGVL>, 2025. Accessed: 2025-09-04.
- Jyothish Pari, Nur Muhammad Shafiullah, Sridhar Pandian Arunachalam, and Lerrel Pinto. The surprising effectiveness of representation learning for visual imitation. *arXiv preprint arXiv:2112.01511*, 2021.
- Shivansh Patel, Xinchun Yin, Wenlong Huang, Shubham Garg, Hooshang Nayyeri, Li Fei-Fei, Svetlana Lazebnik, and Yunzhu Li. A real-to-sim-to-real approach to robotic manipulation with vlm-generated iterative keypoint rewards. *arXiv preprint arXiv:2502.08643*, 2025. URL <https://arxiv.org/abs/2502.08643>.
- Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE international conference on robotics and automation (ICRA)*, pp. 3803–3810. IEEE, 2018.
- Viorica Pătrăucean, Lucas Smaira, Ankush Gupta, Adrià Recasens Contente, Larisa Markeeva, Dylan Banarse, Skanda Koppula, Joseph Heyward, Mateusz Malinowski, Yi Yang, Carl Doersch, Tatiana Matejovicova, Yury Sulsky, Antoine Miech, Alex Frechette, Hanna Klimczak, Raphael Koster, Junlin Zhang, Stephanie Winkler, Yusuf Aytar, Simon Osindero, Dima Damen, Andrew Zisserman, and João Carreira. Perception test: A diagnostic benchmark for multimodal video models, 2023. URL <https://arxiv.org/abs/2305.13786>.
- Ilija Radosavovic, Tete Xiao, Stephen James, Pieter Abbeel, Jitendra Malik, and Trevor Darrell. Real-world robot learning with masked visual pre-training. In *Conference on Robot Learning (CoRL)*, 2022.
- Ilija Radosavovic, Baifeng Shi, Letian Fu, Ken Goldberg, Trevor Darrell, and Jitendra Malik. Robot learning with sensorimotor pre-training. *arXiv preprint arXiv:2306.10007*, 2023.
- Aravind Rajeswaran, Sarvejit Ghotra, Balaraman Ravindran, and Sergey Levine. Epopt: Learning robust neural network policies using model ensembles. *arXiv preprint arXiv:1610.01283*, 2016.
- Martin Riedmiller, Thomas Gabel, Roland Hafner, and Sascha Lange. Reinforcement learning for robot soccer. *Autonomous Robots*, 27(1):55–73, 2009.

- Juan Rocamonde, Victoriano Montesinos, Elvis Nava, Ethan Perez, and David Lindner. Vision-language models are zero-shot reward models for reinforcement learning. *arXiv preprint arXiv:2310.12921*, 2023.
- Erick Rosete-Beas, Oier Mees, Gabriel Kalweit, Joschka Boedecker, and Wolfram Burgard. Latent plans for task agnostic offline reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2022.
- Gautam Salhotra, I-Chun Arthur Liu, Marcus Dominguez-Kuhne, and Gaurav S. Sukhatme. Learning deformable object manipulation from expert demonstrations. *IEEE Robotics and Automation Letters*, 7(4):8775–8782, 2022.
- Saumya Saxena, Mohit Sharma, and Oliver Kroemer. Multi-resolution sensing for real-time control with vision-language models. In *7th Annual Conference on Robot Learning*, 2023. URL <https://openreview.net/forum?id=WuBv9-IGDUA>.
- Pierre Sermanet, Kelvin Xu, and Sergey Levine. Unsupervised perceptual rewards for imitation learning. *arXiv preprint arXiv:1612.06699*, 2016.
- Lin Shao, Toki Migimatsu, Qiang Zhang, Karen Yang, and Jeannette Bohg. Concept2robot: Learning manipulation concepts from instructions and human demonstrations. In *Proceedings of Robotics: Science and Systems (RSS)*, 2020.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Anukriti Singh, Amisha Bhaskar, Peihong Yu, Souradip Chakraborty, Ruthwik Dasyam, Amrit Bedi, and Pratap Tokekar. Varp: Reinforcement learning from vision–language model feedback with agent-regularized preferences. *arXiv preprint arXiv:2503.13817*, 2025. URL <https://arxiv.org/pdf/2503.13817>.
- Laura Smith, J Chase Kew, Xue Bin Peng, Sehoon Ha, Jie Tan, and Sergey Levine. Legged robots that keep on learning: Fine-tuning locomotion policies in the real world. In *2022 international conference on robotics and automation (ICRA)*, pp. 1593–1599. IEEE, 2022a.
- Laura Smith, Ilya Kostrikov, and Sergey Levine. A walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning. *arXiv preprint arXiv:2208.07860*, 2022b.
- Sumedh Sontakke, Jesse Zhang, Séb Arnold, Karl Pertsch, Erdem Biyik, Dorsa Sadigh, Chelsea Finn, and Laurent Itti. Roboclip: One demonstration is enough to learn robot policies. *Advances in Neural Information Processing Systems*, 36, 2024.
- Huajie Tan, Sixiang Chen, Yijie Xu, Zixiao Wang, Yuheng Ji, Cheng Chi, Yaoyu Lyu, Zhongxia Zhao, Xiansheng Chen, Peterson Co, Shaoxuan Xie, Guocai Yao, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. Robo-dopamine: General process reward modeling for high-precision robotic manipulation. *arXiv preprint arXiv:2512.23703*, 2025.
- Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pp. 23–30. IEEE, 2017.
- Sreyas Venkataraman, Yufei Wang, Ziyu Wang, Zackory Erickson, and David Held. Real-world offline reinforcement learning from vision language model feedback. *arXiv preprint arXiv:2411.05273*, 2024. URL <https://arxiv.org/abs/2411.05273>.
- David Venuto, Sami Nur Islam, Martin Klissarov, Doina Precup, Sherry Yang, and Ankit Anand. Code as reward: Empowering reinforcement learning with vlms. *arXiv preprint arXiv:2402.04764*, 2024.

- Jörn Vogel, Annette Hagengruber, Maged Iskandar, Gabriel Quere, Ulrike Leipscher, Samuel Bustamante, Alexander Dietrich, Hannes Hoepfner, Daniel Leidner, and Alin Albu-Schäffer. Edan - an emg-controlled daily assistant to help people with physical disabilities. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- Andrew Wagenmaker, Mitsuhiko Nakamoto, Yunchu Zhang, Seohong Park, Waleed Yagoub, Anusha Nagabandi, Abhishek Gupta, and Sergey Levine. Steering your diffusion policy with latent space reinforcement learning. *arXiv preprint arXiv:2506.15799*, 2025.
- Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Vuong, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning (CoRL)*, 2023.
- Yufei Wang, Zhanyi Sun, Jesse Zhang, Zhou Xian, Erdem Biyik, David Held, and Zackory Erickson. RL-vlm-f: Reinforcement learning from vision-language foundation model feedback. *arXiv preprint arXiv:2402.03681*, 2024. URL <https://arxiv.org/abs/2402.03681>.
- Ge Yan, Kris Wu, and Xiaolong Wang. Ucsd kitchens dataset. <https://vis-www.cs.umich.edu/ucsd-kitchens>, 2023.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Daniel Yang, Davin Tjia, Jacob Berg, Dima Damen, Pulkit Agrawal, and Abhishek Gupta. Rank2reward: Learning shaped reward functions from passive video. *arXiv preprint arXiv:2404.14735*, 2024a. URL <https://arxiv.org/abs/2404.14735>.
- Jingyun Yang, Max Sobol Mark, Brandon Vu, Archit Sharma, Jeannette Bohg, and Chelsea Finn. Robot fine-tuning made easy: Pre-training rewards and policies for autonomous real-world reinforcement learning, 2023.
- Yanting Yang, Minghao Chen, Qibo Qiu, Jiahao Wu, Wenxiao Wang, Binbin Lin, Ziyu Guan, and Xiaofei He. Adapt2reward: Adapting video-language models to generalizable robotic rewards via failure prompts. *arXiv preprint arXiv:2407.14872*, 2024b. URL <https://arxiv.org/abs/2407.14872>.
- Michihiro Yasunaga, Luke Zettlemoyer, and Marjan Ghazvininejad. Multimodal reward-bench: Holistic evaluation of reward models for vision-language models. *arXiv preprint arXiv:2502.14191*, 2025. URL <https://arxiv.org/abs/2502.14191>.
- Runhao Zeng, Dingjie Zhou, Qiwei Liang, Junlin Liu, Hui Li, Changxin Huang, Jianqiang Li, Xiping Hu, and Fuchun Sun. Video2reward: Generating reward function from videos for legged robot behavior learning. *arXiv preprint arXiv:2412.05515*, 2024. URL <https://arxiv.org/abs/2412.05515>.
- Jiahui Zhang, Yusen Luo, Abrar Anwar, Sumedh A. Sontakke, Joseph J. Lim, Jesse Thomason, Erdem Biyik, and Jesse Zhang. Rewind: Language-guided rewards teach robot policies without new demonstrations. *arXiv preprint arXiv:2505.10911*, 2025a. URL <https://arxiv.org/abs/2505.10911>.
- Yi-Fan Zhang, Huanyu Zhang, Haochen Tian, Chaoyou Fu, Shuangqing Zhang, Junfei Wu, Feng Li, Kun Wang, Qingsong Wen, Zhang Zhang, Liang Wang, Rong Jin, and Tieniu Tan. Mmc-realworld: Could your multimodal llm challenge high-resolution real-world scenarios that are difficult for humans?, 2025b. URL <https://arxiv.org/abs/2408.13257>.

- Zijian Zhang, Kaiyuan Zheng, Zhaorun Chen, Joel Jang, Yi Li, Siwei Han, Chaoqi Wang, Mingyu Ding, Dieter Fox, and Huaxiu Yao. Grape: Generalizing robot policy via preference alignment. *arXiv preprint arXiv:2411.19309*, 2024.
- Gaoyue Zhou, Victoria Dean, Mohan Kumar Srirama, Aravind Rajeswaran, Jyothish Pari, Kyle Hatch, Aryan Jain, Tianhe Yu, Pieter Abbeel, Lerrel Pinto, Chelsea Finn, and Abhinav Gupta. Train offline, test online: A real robot learning benchmark, 2023.
- Henry Zhu, Justin Yu, Abhishek Gupta, Dhruv Shah, Kristian Hartikainen, Avi Singh, Vikash Kumar, and Sergey Levine. The ingredients of real-world robotic reinforcement learning. *arXiv preprint arXiv:2004.12570*, 2020.
- Yifeng Zhu, Abhishek Joshi, Peter Stone, and Yuke Zhu. Viola: Imitation learning for vision-based manipulation with object proposal priors. In *Conference on Robot Learning (CoRL)*, 2022a.
- Yifeng Zhu, Peter Stone, and Yuke Zhu. Bottom-up skill discovery from unsegmented demonstrations for long-horizon robot manipulation. *IEEE Robotics and Automation Letters*, 7(2):4126–4133, 2022b.

A ROBOREWARD DATASET

A.1 DATASET SOURCES

Table 4: The various dataset sources comprising the RoboReward data mixture and benchmark. The resulting corpus contains 54,135 examples in total: 45,072 train, 6,232 validation, and 2,831 test.

Name	Embodiment	Description	Perspective	Train	Val	Test	Citation
RoboArena	DROID (Franka-based)	Distributed real-world evaluation episodes with per-episode progress scores and pairwise preferences.	Mix	7337	1000	1000	Atreya et al. (2025)
Berkeley Bridge	WidowX	The robot interacts with household environments including kitchens, sinks, and tabletops. Skills include object rearrangement, sweeping, stacking, folding, and opening/closing doors and drawers.	Exocentric	3826	496	100	Walke et al. (2023)
Freiburg Franka Play	Franka	The robot interacts with toy blocks, it pick and places them, stacks them, unstacks them, opens drawers, sliding doors and turns on LED lights by pushing buttons.	Egocentric	2856	352	91	Rosete-Beas et al. (2022); Mees et al. (2023)
USC Jaco Play	Jaco 2	The robot performs pick-place tasks in a tabletop toy kitchen environment.	Exocentric	2428	417	100	Dass et al. (2023)
Roboturk	Sawyer	Sawyer robots flattens laundry, builds towers from bowls and searches objects.	Exocentric	1463	0	97	Mandlekar et al. (2019)
NYU VINN	Hello Stretch	The robot opens cabinet doors for a variety of cabinets.	Egocentric	2369	0	0	Pari et al. (2021)
Austin VIOLA	Franka	The robot performs various household-like tasks, such as setting up the table, or making coffee using a coffee machine.	Exocentric	167	239	60	Zhu et al. (2022a)
Berkeley Autolab UR5	UR5	The data consists of 4 robot manipulation tasks: simple pick-and-place of a stuffed animal between containers, sweeping a cloth, stacking cups, and a more difficult pick-and-place of a bottle that requires precise grasp and 6 DOF rotation.	Egocentric	2388	430	100	Chen et al.
TOTO	Franka	The TOTO Benchmark Dataset contains trajectories of two tasks: scooping and pouring. For scooping, the objective is to scoop material from a bowl into the spoon. For pouring, the goal is to pour some material into a target cup on the table.	Exocentric	2986	0	0	Zhou et al. (2023)
NYU ROT	xArm	The robot arm performs diverse manipulation tasks on a tabletop such as box opening, cup stacking, and pouring, among others.	Exocentric	35	8	0	Haldar et al. (2023)
Stanford HYDRA	Franka	The robot performs the following tasks in corresponding environment: making a cup of coffee using the keurig machine; making a toast using the oven; sorting dishes onto the dish rack.	Exocentric	507	203	91	Belkhole et al. (2023)
Austin BUDS	Franka	The robot is trying to solve a long-horizon kitchen task by picking up pot, placing the pot in a plate, and push them together using a picked-up tool.	Exocentric	127	0	0	Zhu et al. (2022b)
UCSD Kitchen	xArm	The dataset offers a comprehensive set of real-world robotic interactions, involving natural language instructions and complex manipulations with kitchen objects.	Exocentric	393	122	95	Yan et al. (2023)
UCSD Pick Place	xArm	The robot performs pick and place tasks in table top and kitchen scenes. The dataset contains a variety of visual variations.	Exocentric	2384	0	100	Feng et al. (2023)
Austin Sirius	Franka	The dataset comprises two tasks, kcup and gear. The kcup task requires opening the kcup holder, inserting the kcup into the holder, and closing the holder. The gear task requires inserting the blue gear onto the right peg, followed by inserting the smaller red gear.	Exocentric	1355	0	87	Liu et al. (2023)
Tokyo PR2 Fridge Opening	PR2	PR2 opening/closing fridge and related appliance interactions.	Exocentric	357	0	0	Oh et al. (2023)
Tokyo PR2 Tabletop Manipulation	PR2	Reaching, grasping, placing on PR2 across varied objects and scenes.	Exocentric	293	325	73	Oh et al. (2023)
UTokyo xArm Pick-Place	xArm	The robot picks up a white plate, and then places it on the red plate.	Egocentric	301	0	0	Matsushima et al. (2023)
UTokyo xArm Bimanual	Dual xArms	The robots reach a towel on the table. They also unfold a wrinkled towel.	Egocentric	161	0	71	Matsushima et al. (2023)
Berkeley MVP	xArm	Basic motor control tasks (reach, push, pick) on table top and toy environments (toy kitchen, toy fridge).	Egocentric	1218	228	71	Radosavovic et al. (2022)
Berkeley RPT	Franka	Picking, stacking, destacking, and bin picking with variations in objects.	Egocentric	1069	364	86	Radosavovic et al. (2023)
KAIST Nonprehensile Objects	Franka	The robot performs various non-prehensile manipulation tasks in a tabletop environment. It translates and reorients diverse real-world and 3d-printed objects to a target 6 dof pose.	Exocentric	406	162	53	Salhotra et al. (2022)
LSMO	Cobotta	The robot avoids obstacle on the table and reaches the target object.	Exocentric	97	0	71	
CMU Franka Pick-Insert	Franka	The robot tries to pick up different-shaped objects placed in front of it. It also tries to insert particular objects into a cylindrical peg.	Exocentric	1193	374	83	Saxena et al. (2023)
Berkeley Fanuc Manipulation	Fanuc	A Fanuc robot performs various manipulation tasks. For example, it opens drawers, picks up objects, closes doors, closes computers, and pushes objects to desired locations.	Exocentric	860	287	91	Radosavovic et al. (2023)
CMU Play Fusion	Franka	The robot plays with 3 complex scenes: a grill with many cooking objects like toaster, pan, etc. It has to pick, open, place, close. It has to set a table, move plates, cups, utensils. And it has to place dishes in the sink, dishwasher, hand cups etc.	Exocentric	1269	358	92	Lynch et al. (2023)
DROID	Franka	Various household manipulation tasks	Exocentric	3071	378	100	Khazatsky et al. (2024)
RT-1 Robot Action	Google Robot	Robot picks, places and moves 17 objects from the google micro kitchens.	Exocentric	3988	461	100	Brohan et al. (2022)
DLR Wheelchair Shared Control	DLR EDAN	The robot grasps a set of different objects in a table top and a shelf.	Exocentric	168	28	19	Vogel et al. (2020)

A.2 DATA CLEANING AND AUGMENTATION DETAILS

For a successful episode $e = (v, t, r)$ with $r = 5$, we construct additional training triples in two ways: (i) counterfactual commands $\tilde{t}$ paired with calibrated labels $\tilde{r} \in \{1, 2, 3, 4\}$ for the same video v , and (ii) clipped videos $\tilde{v}$ paired with labels $\tilde{r} \in \{1, 2, 3, 4\}$ for the original command t . For counterfactual relabeling, we use a multi-stage generation pipeline: (1) GPT-5 mini performs video analysis, (2) GPT-5 mini produces a plan for distinct failure modes that enforce a strict ordering $1 < 2 < 3 < 4 < 5$, (3) Qwen3 generates one imperative command per score in sequence, and (4) GPT-5 mini validates the resulting set and rejects it if it is inconsistent, triggering regeneration. For negative clipping, we generate a small ladder of clipped rollouts per episode and validate the resulting score assignments in the same way. Overall, this augmentation converts success videos into a balanced ladder of outcomes without fabricating videos and expands our training corpus.

A.2.1 PROMPT REWRITE (INVARIANT CLEAN-UP)

Model. Qwen3-4B-Instruct-2507 (text-only).

Purpose. Correct grammar/spelling while preserving semantics (e.g., fix “*palce dishes in the dish rack*” to “*place the dishes in the dish rack*”).

Prompt.

Rewrite the following task description to correct grammar and spelling only.
Fix capitalization errors by beginning a sentence or phrase with a capital letter.
Do not change meaning.
Task description: {TASK}
Return only the corrected text.

A.2.2 NEGATIVE EXAMPLE GENERATION

Models. We use gpt-5-mini-2025-08-07 for video analysis and planning, and Qwen3-4B-Instruct-2507 for command generation.

Rubric (end-of-episode).

Rubric for end-of-episode progress (judge only the final state without time limits):
1 - No Success: Final state shows no goal-relevant change for the command.
2 - Minimal Progress: Final state shows a small but insufficient change toward the goal.
3 - Partial Completion: The final state shows good progress toward the goal but violates more than one requirement or a major requirement.
4 - Near Completion: Final state is correct in region and intent but misses a single minor requirement.
5 - Perfect Completion: Final state satisfies all requirements.

Video Analysis (VL). We sample the rollout video at 1 FPS and provide the resulting image frames (including the final frame) along with the original task description.

You are analyzing a video of a robot performing a short-horizon manipulation task.
Describe the starting scene and objects visible. Be sure to mention objects in the task description.
Describe thoroughly object positions, locations and their relations to each other and to the robot for the starting scene.
Use the ROBOT BASE perspective for left/right; if the base is not visible, use the CAMERA perspective.
Then describe, step by step, what the robot does from start to end, focusing on the final state.

Be concrete and factual. Do not invent objects that are not visible.
Pay special attention to the last frame, especially when describing the final state.

Task description: {ORIGINAL_TASK}

Output sections:

- 1) Scene and objects
- 2) Robot actions step by step
- 3) Final state summary including the final state of the robot and relevant objects given the task description.

Planning (Text).

Plan carefully and step by step.

Goal: Think step by step to design distinct failure modes and concrete ideas for new task commands for scores 1, 2, 3, and 4, so that $1 < 2 < 3 < 4 < 5$, where 5 is the original task fully satisfied by the video.

Judge only the final state and ignore time limits.

Use only visible objects and relations grounded in the video analysis.
Each score must correspond to a strictly closer final state to success than the previous one.

For reward 1, the robot performed no relevant action towards the goal (e.g., handled a completely different object than the one in the task description). Assign a distinct failure mode to each of 2, 3, and 4.

The ideas must not be entailed by or easier than the original task.

Justify each idea's score in parentheses.

Critical: ideas for different scores must NOT be paraphrases or trivial rewrites of each other.

Each idea must NOT already be satisfied by the starting scene described above; require a visible change from the initial state.

Use the initial-state description explicitly to avoid proposing goals that are already true at the initial state.

For lower scores 2, 3, and 4 specifically, do NOT use control-action phrases similar to 'release the gripper' or 'let go of the gripper' or 'not in contact with the robot'.

For scores 3 and 4, refer to the SAME primary target object/entity using the same name as in the original task description. Do not switch to a different instance (e.g., 'cabinet' -> 'upper cabinet door').

Match the diction, writing style and complexity of the original task description.

Original task (score 5): {ORIGINAL_TASK}

Video analysis:

{VIDEO_ANALYSIS}

Rubric:

{RUBRIC}

Clarification for scoring 3 vs 4 (MINOR vs MAJOR requirements):

- Treat a requirement as MINOR when the PRIMARY object identity and PRIMARY spatial relation to the reference object are satisfied, but an AUXILIARY constraint is missed.

- Examples of AUXILIARY constraints that can be MINOR: remaining on the same support surface (e.g., keeping a cloth directly under the object), releasing/holding the gripper at the end, small orientation or placement tolerances, or cosmetic positioning details that do not change the core relation.

- Therefore, if the target object is correctly placed relative to the specified reference and all core elements are correct, but an auxiliary constraint like keeping the original cloth under the object is missed, prefer score 4 (Near Completion) rather than 3 (Partial Completion).

- Use score 3 only when multiple constraints are missed or when the missed constraint is CENTRAL to the task identity (e.g., primary spatial relation is incorrect).

Produce the following sections in order:

- 1) Reasoning - step-by-step analysis of the scene and of what final-state properties define success for the original task 5.

- 2) Separation plan - explain how to construct 1, 2, 3, and 4 so that $1 < 2 < 3 < 4 < 5$ using only visible, concrete constraints.
- 3) Ideas for new task commands - propose highly focused candidate commands for each of scores 1, 2, 3, and 4. Justify each with a one-line reason for why it receives that score.
- 4) Monotonicity check - for each adjacent pair (1<2), (2<3), (3<4), and (4<5), justify why the later command is strictly closer to success under the rubric.
- 5) Final set of suggested commands - list the best final commands given the reasoning and separation plan for scores 1, 2, 3, 4, and 5 (remember 5 is the original task). Output only the commands with their scores, one per line.

Command Generation (Text, one score at a time).

Using the final suggested commands, generate a single imperative task command (one line) for the SAME video such that:

- Its correct evaluation under the rubric would be score {K} for the final state shown in the video.
- The command is stricter or different from the original so that the same video does not fully satisfy it if score is below 5.
- It is not entailed by the original task and is not an easier subset.
- Do not mention or reference the original task for the new task description.
- It uses only visible objects and relations described in the video analysis.
- Use plain ASCII characters only.
- Keep under 25 words.
- Start with a verb to make it an imperative command and do not mention the score or any meta-instructions in the command.

Original task (score 5): {ORIGINAL_TASK}

Rubric:
{RUBRIC}

Final suggested commands:
{PLAN_TEXT}

Keep in mind the following previously generated commands for lower scores and avoid duplication or contradiction while maintaining strict ordering (lower scores farther from success than higher scores):
{HISTORY}

Now output only the command for score {K} on a single line with no quotes and no extra commentary.

A.2.3 VERIFICATION

Model. gpt-5-mini-2025-08-07.

Purpose. Verify each example by checking (i) the task description is coherent and grounded in the video, and (ii) the provided discrete reward score matches the rubric. We keep only examples that pass this check.

Inputs. We sample each rollout video at 1 FPS and pass the resulting image frames (including the true final frame) along with the task description and the provided score.

Prompt.

You are a strict annotation validator for short-horizon ROBOT TASK videos.

INPUTS

- TASK DESCRIPTION: {TASK_DESCRIPTION}
- PROVIDED SCORE: {PROVIDED_SCORE} (must be one of 1,2,3,4,5)

YOUR JOB (single-pass):

- 1) Watch the video.
- 2) Using the rubric below, evaluate the robot's actions during the task and the final state, and determine whether the PROVIDED SCORE is correct.
- 3) Check that the TASK DESCRIPTION is coherent and refers to the correct items/actions visible in the video.
- 4) Decision basis and rejection criteria:
 - If the TASK DESCRIPTION is incoherent with the video (e.g., wrong primary object identity, inconsistent relation, key details not visible, or rule conflicts), you MUST output ANSWER: FALSE.
 - Otherwise, output ANSWER: TRUE if the PROVIDED SCORE matches the rubric for the video; output ANSWER: FALSE if it does not match.

RUBRIC (end-of-episode, discrete progress):

- 1 - No Success: Final state shows no goal-relevant change for the command (e.g., wrong object was handled)
- 2 - Minimal Progress: Final state shows a small but insufficient change toward the goal.
- 3 - Partial Completion: The final state shows good progress toward the goal but violates more than one requirement or a major requirement.
- 4 - Near Completion: The final state is correct in region and intent but misses a single minor requirement.
- 5 - Perfect Completion: Fully satisfies the task as described; ALL requirements met.

MINOR VS MAJOR REQUIREMENTS (clarification for 3 vs 4):

- Treat a requirement as MINOR when the PRIMARY object identity and PRIMARY spatial relation to the reference object are satisfied, but an AUXILIARY constraint is missed.
- Examples of AUXILIARY constraints that can be MINOR: remaining on the same support surface (e.g., keeping a cloth directly under the object), holding/releasing the gripper at the end, small orientation or placement tolerances, or cosmetic positioning details that do not change the core relation.
- Therefore, if the target object is correctly placed relative to the specified reference and all core elements are correct, but an auxiliary constraint like keeping the original cloth under the object is missed, prefer score 4 (Near Completion) rather than 3 (Partial Completion).
- Use score 3 only when multiple constraints are missed or when the missed constraint is CENTRAL to the task identity (e.g., wrong object identity or the primary spatial relation is incorrect).

PERSPECTIVE & COMMON PITFALLS (apply strictly):

- Left/Right: Use the ROBOT BASE perspective. If the base is not visible, use the CAMERA perspective.
- Object naming: REJECT if the description mislabels key items.
- Consistency rules:
 - * If the primary target object/entity name in the TASK DESCRIPTION does not match the object named in the video, treat as mismatch -> REJECT. Do NOT treat category synonyms or toy/real variants as equivalent (e.g., "sandwich slice" vs "toy cheese").
 - * End location naming: Allow common surface synonyms (e.g., towel/cloth/napkin/rag) as equivalent IF color and relative placement match, and there is no conflicting evidence. If color or relative placement clearly differ, REJECT.
- Irrelevant objects: Ignore objects not needed to perform the task (do NOT reject just for extra/unused items).
- Move vs slide: Do not conflate sliding on a surface with picking up/placing if the description depends on this distinction.
- Color ambiguity: If color terms feel ambiguous BUT the intended item/action is still unambiguous for executing the task, do NOT reject for color alone.
- Visibility: If key details to judge success are not visible or the video is too unclear, REJECT.

TASK SANITY CHECKS (MUST REJECT if any apply):

- Task string is empty, placeholder, or nonsense and not a valid task description/command (e.g., "No image loaded in this hit").
- Task does not reference any visible objects/relations in the video or cannot be grounded.

OUTPUT FORMAT (STRICT)

- 1) First, write your reasoning (few sentences) that clearly states whether the PROVIDED SCORE matches the rubric and why.
- 2) Then on a new line write exactly ONE of the following without any formatting and nothing else:

ANSWER: TRUE

or

ANSWER: FALSE

A.3 HUMAN VERIFICATION FOR ROBOREWARDBENCH

To construct a higher-trust evaluation suite, we additionally perform *human* verification for the test split. Concretely, each example is reviewed by one human annotator, who is asked to confirm that the end-of-episode reward label is justified under our rubric, given the rollout video and task description (see example at Figure 6). When a mismatch is found, we discard the example. We then subsample from the remaining verified examples to form a clean evaluation set. The resulting human-verified test split contains 2,831 examples, which we refer to as **RoboRewardBench**.

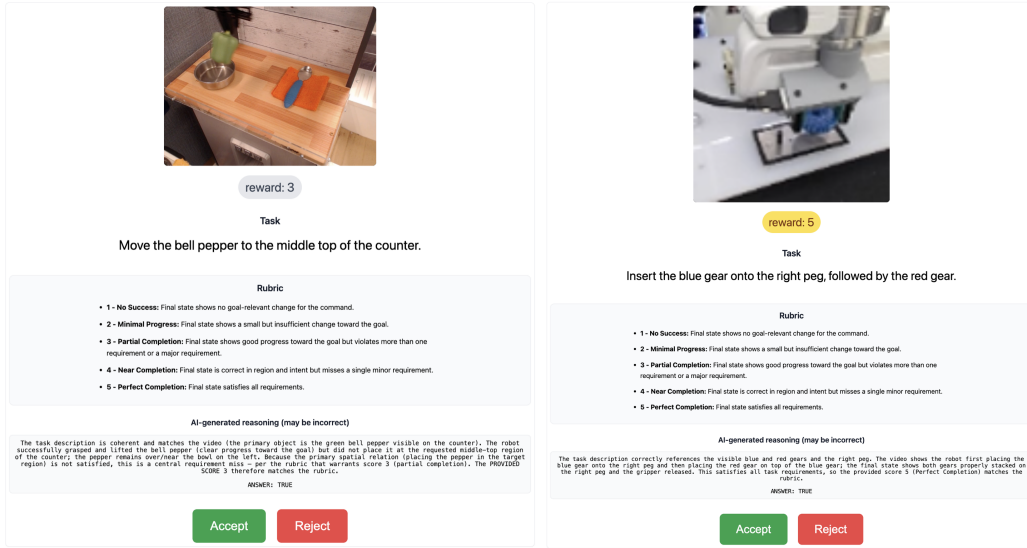


Figure 6: Annotation UI used for human verification. Annotators watch the rollout video and are shown the task text, the provided reward label, the rubric, and the GPT-5 mini verification rationale produced by our automated validation step. They then accept or reject the example based on whether the reward label is justified by the video under the rubric.

B EXPERIMENTAL DETAILS

B.1 ROBOMIMIC EXPERIMENTS

For all Robomimic experiments, we run DSRL-NA with the hyperparameters as specified for each environment in Wagenmaker et al. (2025). For completeness, we include these in Tables 5 and 6.

Table 5: Common DSRL hyperparameters for Robomimic experiments.

Hyperparameter	Value
Learning rate	0.0003
Batch size	256
Activation	Tanh
Target entropy	0
Target update rate (τ)	0.005
Number of actor and critic layers	3
Number of critics	2
Number of environments	4

Table 6: Hyperparameters for DSRL Robomimic experiments.

Hyperparameter	Lift	Can	Square
Action chunk size	4	4	4
Hidden size	2048	2048	2048
Gradient steps per update	30	20	20
Q^W update steps	10	10	10
Discount factor	0.99	0.99	0.999
Action magnitude	1.5	1.5	1.5
Initial steps	24000	24000	32000
Train denoising steps	20	20	100
Inference denoising steps	8	8	8

B.2 BENCHMARKING WITH ROBOREWARDBENCH

Table 7: Vision–language models evaluated on **RoboRewardBench** and their overall results. Rows are ordered by overall group-wise mean absolute error (MAE; lower is better). *Limited* denotes models available only via a restricted API at the time of evaluation, for which little public technical information is available (e.g., parameter count).

Rank	Model	Creator	Parameters	Access	MAE	Reference
1	RoboReward (8B)	Ours	8B	Open	0.665	This work
2	GPT-5 mini (2025-08-07)	OpenAI	–	Limited	0.691	OpenAI (2025c)
3	GPT-5 (2025-08-07)	OpenAI	–	Limited	0.811	OpenAI (2025c)
4	RoboReward (4B)	Ours	4B	Open	0.845	This work
5	Gemini 3 Pro (Preview)	Google DeepMind	–	Limited	0.851	Google DeepMind (2025d)
6	GPT-5.2 (2025-12-11)	OpenAI	–	Limited	0.887	OpenAI (2025b)
7	Qwen3-VL Instruct (8B)	Alibaba	8B	Open	0.892	Bai et al. (2025a)
8	GPT-5.1 (2025-11-13)	OpenAI	–	Limited	0.901	OpenAI (2025a)
9	Gemini 2.5 Pro	Google DeepMind	–	Limited	0.902	Google DeepMind (2025b)
10	Qwen3-VL Instruct (30B)	Alibaba	30B	Open	0.903	Bai et al. (2025a)
11	Gemini Robotics-ER 1.5	Google DeepMind	–	Limited	0.906	Google DeepMind (2025e)
12	Gemini 3 Flash (Preview)	Google DeepMind	–	Limited	0.917	Google DeepMind (2025c)
13	Gemini 2.5 Flash	Google DeepMind	–	Limited	0.943	Google DeepMind (2025b)
14	Gemini 2.5 Flash-Lite	Google DeepMind	–	Limited	0.990	Google DeepMind (2025a)
15	Qwen2.5-VL Instruct (72B)	Alibaba	72B	Open	0.991	Bai et al. (2025b)
16	Qwen3-VL Instruct (4B)	Alibaba	4B	Open	1.032	Bai et al. (2025a)
17	Qwen2.5-VL Instruct (32B)	Alibaba	32B	Open	1.137	Bai et al. (2025b)
18	Qwen2.5-VL Instruct (7B)	Alibaba	7B	Open	1.172	Bai et al. (2025b)
19	Llama 4 Maverick Instruct	Meta	–	Open	1.271	Meta (2025)
20	GPT-5 nano (2025-08-07)	OpenAI	–	Limited	1.295	OpenAI (2025c)
21	Llama 4 Scout Instruct	Meta	–	Open	1.485	Meta (2025)
22	Qwen2.5-VL Instruct (3B)	Alibaba	3B	Open	1.607	Bai et al. (2025b)

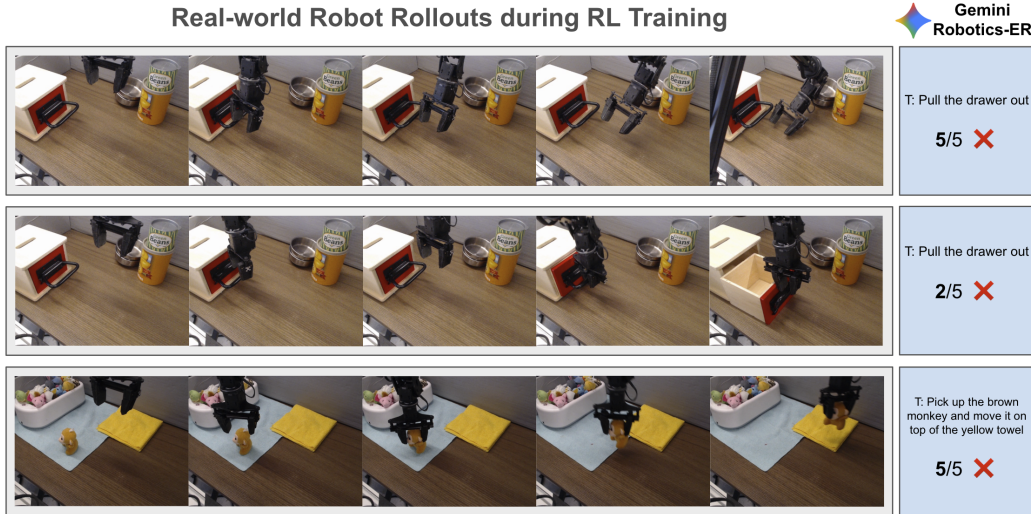


Figure 7: **Failure cases of Gemini Robotics-ER 1.5 as a reward model on real robot rollouts.** Each row shows a rollout (frames left) and the model’s end-of-episode progress score given the video rollout and task (right). **Top:** *Pull the drawer out.* The robot collapses to the right of the handle and the drawer remains closed, but the model predicts **5/5** (false positive). **Middle:** *Pull the drawer out.* The robot successfully pulls the drawer open, but the model predicts **2/5** (false negative). **Bottom:** *Pick up the brown monkey and move it on top of the yellow towel.* The final frame shows the monkey held above the towel (obviously hovering rather than clearly placed), yet the model predicts **5/5**, showing a lack of simple spatial reasoning.

B.3 THE PITFALLS OF STATE-OF-THE-ART VLMS AS REWARD MODELS

Our benchmarking results (Section 5.1) and real-world RL experiments (Section 5.2) show that frontier VLMS can act as reward models, but they still make simple mistakes that are easy for humans to catch. In this section, we provide qualitative examples that help explain these failure modes.

We focus on **Gemini Robotics-ER 1.5** because it is a frontier model trained on robotics data and designed for embodied reasoning and progress estimation (Google DeepMind, 2025f). Despite these targeted capabilities, it still produces incorrect progress scores on real robot rollouts, including during RL training.

Gemini Robotics-ER 1.5 makes high-impact reward mistakes. Figure 7 shows three representative errors. For row 1, the task is *pull the drawer out*. The robot does not successfully grasp the handle, the drawer remains closed, and the gripper falls over to the right. Nevertheless, Gemini Robotics-ER assigns a score of 5/5. For row 2, the robot this time succeeds in pulling the drawer open, but Gemini Robotics-ER assigns only a 2/5 instead of a perfect completion score of 5, leading to a false negative. Finally, for row 3, the task is *pick up the brown monkey and move it on top of the yellow towel*. The final frame shows the monkey held above the towel (not clearly placed), yet the model deems the trajectory ends with a perfect completion, assigning a reward of 5/5. These kinds of false negatives and positives are particularly problematic when assigning rewards during RL: over-rewarding failures can reinforce the wrong behavior, and under-rewarding successes can weaken the learning signal.

To better understand these mis-scored rewards and verify whether the model is hallucinating key events, we prompt Gemini Robotics-ER 1.5 “Describe the video in great detail.” with default hyperparameters:

Row 1: "Pull the drawer out" (model assigns 5/5 on failure)

Based on the video frames, here is a detailed description of the scene and action:

The video begins with a static shot of a wooden table surface. On the left side of the frame sits a small, light-colored wooden box. The front face of the box has a red panel with a black metal handle attached, featuring a looped design. In the background, to the right of the box, are several items: a stack of containers consisting of a yellow canister at the bottom and a green can labeled "Green Beans" on top, and a small, nested metal bowl next to them.

Starting at 00:01, a black robotic arm, specifically its end-effector or gripper, moves into the frame from the top right. The gripper is articulated with two fingers. It descends towards the wooden box on the left.

From 00:03 to 00:04, the robot arm positions the open gripper directly in front of the black handle on the box. The fingers of the gripper are carefully aligned with the handle.

By 00:05, the gripper's fingers close around the black handle, securing its grasp on the box.

Finally, from 00:06 to 00:07, the robot arm pulls back slightly, dragging the wooden box with it by the handle. A part of the robot's vertical structure briefly appears on the far left edge of the frame. The action demonstrates the robot's ability to identify and manipulate objects using the handle.

Gemini Robotics-ER 1.5 correctly describes the scene: it correctly identifies the main objects (the wooden box with a red front panel and black handle, the stacked canisters labeled "Green Beans," and the metal bowls) and provides a plausible high-level intent of the action (the arm approaching the handle). However, the description incorrectly asserts a precise grasp ("fingers close around the handle") and a successful pull ("dragging the wooden box") even though the rollout does not clearly show secure contact or drawer motion. This suggests a failure mode where the model interpolates a typical manipulation narrative from partial visual evidence, rather than tracking the true temporal progression and fine-grained spatial state needed for reward assignment.

Row 2: "Pull the drawer out" (model assigns 2/5 on success)

Based on the sequence of images, here is a detailed description of the scene and action:

The video shows a high-angle view of a robot arm interacting with objects on a wooden tabletop. On the left side of the frame is a light-colored wooden box with a prominent black D-shaped handle attached to a red rectangular plate on its front face. Behind this box, and to the right, are several other items: a stack of two metal bowls, a large yellow cylindrical container, and a smaller green can labeled "Green Beans" sitting on top of the yellow container.

At the beginning of the clip (00:00), a black, two-fingered robot gripper is visible in the upper center part of the frame, poised in the air above the wooden surface.

As the video progresses, the robot arm moves steadily from right to left (00:01-00:02). The gripper arm adjusts its position, lowering itself and rotating to align with the black handle on the wooden box.

By the final frame (00:03), the robot arm's gripper has moved significantly closer and is positioned directly in front of the handle, appearing ready to grasp it. The movement captures the robot arm in the process of targeting and approaching the handle on the wooden box.

The description again captures the static scene well: it correctly identifies the drawer box, the D-shaped handle on the red plate, and the surrounding objects (bowls, yellow container, “Green Beans” can), and it accurately describes the gripper’s approach and alignment behavior. The failure is that it stops short of narrating the decisive task outcome. In the rollout, the drawer is actually pulled open, but the model’s description ends at “appearing ready to grasp,” suggesting it under-tracks late-stage temporal events and state changes.

Row 3: “Pick up the brown monkey and move it on top of the yellow towel” (model assigns 5/5 without a clear placement)

Based on the video frames, a robotic gripper performs a sequence of actions involving a plush toy.

1. ****Initial State (00:00):**** The scene features a small brown monkey plush toy on a light blue mat. To the right of the mat, on a wooden surface, is a folded yellow cloth. A robotic gripper is positioned above the objects.
2. ****Grasping Action (00:01 - 00:04):**** The robotic gripper moves down towards the brown monkey plush toy. The gripper's claws open and descend around the toy. The claws then close, grasping the monkey and lifting it from the blue mat.
3. ****Relocation (00:05 - 00:07):**** The gripper, holding the monkey, lifts it higher and then moves horizontally to the right, carrying the toy over the wooden surface towards the yellow cloth.
4. ****Placement (00:08):**** The gripper positions the brown monkey directly above the folded yellow cloth and lowers it, placing the toy onto the cloth.

The model states a completed placement (“placing the toy onto the cloth”), but the rollout does not clearly show the monkey resting on the towel. Instead, the final frame is ambiguous and consistent with the monkey still being held above the towel. The description, therefore, hallucinates a key success event (the release/placement), which leads to an incorrect **5/5** score.

Takeaway. These examples suggest that even frontier VLMs trained on robotics data can understand the scene at a high level while still failing on the fine-grained spatial and temporal details that reward modeling depends on (e.g., whether a grasp is secure, whether a drawer actually opens, or whether an object is truly placed versus hovering). In practice, both RoboReward 8B and Gemini Robotics-ER 1.5 often produce reasonable rewards, but these kinds of false rewards are enough to contribute to errors on RoboRewardBench (Section 5.1) and create noticeable gaps to oracle human rewards in real-world RL (Section 5.2).

B.4 REAL-WORLD RL EXPERIMENT

DSRL finetunes the behavior of a base diffusion policy by modifying the input noise given to its denoising process. For the base diffusion policy, we utilize the diffusion transformer architecture due to Dasari et al. (2025). We pretrain the diffusion policy on the BridgeData V2 dataset and use goal image conditioning to specify the task. See Table 8 for the hyperparameters used for the diffusion policy.

For DSRL, we utilize the DSRL-SAC variant described in Wagenmaker et al. (2025). We follow the procedure used for the WidowX experiments in Wagenmaker et al. (2025) exactly. In particular, we first roll out the base diffusion policy 20 times to warm-start the replay buffer, and then start running RL training. RL hyperparameters are given in Table 9 (all reward models use the same DSRL hyperparameters).

Table 8: **Hyperparameters for base diffusion policy in WidowX experiments.**

Hyperparameter	Value
Batch size	2048
Learning rate	0.0003
Training steps	100000
LR scheduler	cosine
Warmup steps	2000
Action chunk size	1
Train denoising steps	100
Inference denoising steps	8
Image encoder	ResNet-34
Hidden size	256
Number of Heads	1
Number of Layers	3
Feedforward dimension	512

Table 9: **Hyperparameters for DSRL WidowX experiments.**

Hyperparameter	Value
Hidden size	1024
Gradient steps per update	30
Discount factor	0.97
Action magnitude	1.5

Model	Mean Absolute Error (↓ better)	Robo Arena	Austin Sir- ius	Berkeley Au- tolab UR5	Berkeley Fanuc Ma- nip- ula- tion	Berkeley MVP	Berkeley RPT	Berkeley Bridge	CMU Play Fu- sion	DLR Wheelchari Shared Con- trol	DROID	RT-1 Robot Ac- tion	CMU Franka Pick- Insert	USC Jaco Play	KAIST Non- pre- hen- sile Ob- jects	Roboturk	Stanford HY- DRA	Freiburg Franka Play	LSMO	UCSD Kitchen	UCSD Pick Place	Tokyo PR2 Table- top Ma- nip- ula- tion	UTokyo xArm Bi- man- ual	Austin VI- OLA
RoboReward 8B (ours)	0.665	0.768	0.701	0.340	0.538	0.577	0.674	0.770	0.587	0.474	0.670	0.830	0.807	0.720	0.396	0.485	0.890	0.473	0.493	0.947	0.100	0.452	1.394	1.200
GPT-5 mini (2025-08-07)	0.691	0.862	0.379	0.640	0.769	0.408	0.581	0.530	1.141	0.158	0.640	0.630	0.566	1.090	0.774	0.505	0.835	1.077	0.676	0.832	0.480	0.411	1.592	0.317
GPT-5 (2025-08-07)	0.811	1.028	0.310	0.830	0.934	0.845	1.174	0.790	0.989	0.211	0.760	0.830	1.048	0.830	0.868	0.629	0.670	1.110	0.648	1.063	0.510	0.918	1.366	0.283
RoboReward 4B (ours)	0.845	0.806	0.897	0.700	0.681	0.887	0.721	0.910	0.663	0.579	0.880	0.890	0.843	0.950	0.698	0.969	1.176	0.846	0.535	0.705	0.330	0.562	1.282	1.933
Gemini 3 Pro (Preview)	0.851	1.234	0.379	0.690	0.824	1.394	0.395	0.750	0.630	0.316	0.820	1.040	0.795	0.890	1.491	0.990	0.846	0.989	0.972	1.242	0.370	0.438	1.789	0.283
GPT-5.2 (2025-12-11)	0.887	0.973	0.759	0.780	1.165	0.521	1.407	0.940	1.185	0.263	0.910	0.940	0.506	0.910	1.057	0.557	0.780	1.308	0.986	1.105	0.500	0.822	1.718	0.317
Qwen3-VL Instruct (8B)	0.892	0.847	0.713	0.420	0.890	0.817	0.919	0.980	1.163	0.474	0.890	1.050	1.000	0.950	0.755	0.794	1.000	1.385	1.423	0.895	0.540	0.822	1.085	0.700
GPT-5.1 (2025-11-13)	0.901	1.075	0.575	0.960	0.923	0.676	1.012	1.190	1.000	0.368	1.160	1.000	0.855	0.940	0.906	0.619	1.066	1.242	0.761	1.316	0.570	0.589	1.437	0.483
Gemini 2.5 Pro	0.902	0.936	0.333	0.920	0.736	1.296	0.709	1.170	0.804	0.842	0.920	1.200	0.687	0.890	1.113	1.309	0.495	1.187	0.915	1.316	0.580	0.466	1.746	0.167
Qwen3-VL Instruct (30B)	0.903	0.993	0.632	0.780	1.121	0.465	0.988	1.160	1.337	0.474	1.070	1.040	0.590	1.010	0.849	0.897	1.088	1.099	0.746	1.032	0.610	0.973	1.423	0.383
Gemini Robotics-ER 1.5	0.906	1.002	0.448	0.730	0.791	1.394	0.442	1.000	0.891	0.895	0.760	0.980	0.506	0.980	1.208	1.165	0.846	1.176	0.521	1.326	0.750	0.658	2.070	0.300
Gemini 3 Flash (Preview)	0.917	1.058	0.276	0.550	0.736	1.352	0.430	0.990	0.707	1.053	0.770	1.130	0.819	1.100	1.509	1.134	0.659	1.088	0.549	1.716	0.950	0.767	1.648	0.100
Gemini 2.5 Flash	0.943	1.190	0.655	0.850	0.791	1.254	0.826	1.080	0.783	0.526	0.830	1.430	0.506	0.990	1.358	0.979	0.956	1.341	0.465	1.516	0.760	0.671	1.676	0.250
Gemini 2.5 Flash-Lite	0.990	1.136	0.609	1.110	1.154	0.958	1.221	1.070	1.065	0.368	1.080	1.130	1.012	0.960	0.849	1.021	1.044	1.198	1.197	1.316	0.440	0.863	1.465	0.500
Qwen2.5-VL Instruct (72B)	0.991	0.848	0.575	1.100	1.110	0.437	1.209	1.180	0.739	1.105	0.920	1.270	0.711	1.190	0.887	0.876	1.110	1.264	1.141	1.200	1.060	0.452	2.141	0.267
Qwen3-VL Instruct (4B)	1.032	0.929	1.115	0.680	1.187	1.155	0.930	1.140	1.011	0.895	1.210	1.230	0.807	1.250	1.094	1.113	1.176	1.165	1.099	1.063	0.920	0.959	1.000	0.600
Qwen2.5-VL Instruct (32B)	1.137	1.064	0.782	1.510	1.099	0.775	1.302	1.150	1.033	1.316	1.010	1.220	0.867	1.130	1.151	1.000	1.626	1.308	1.479	1.347	0.910	0.521	1.704	0.850
Qwen2.5-VL Instruct (7B)	1.172	1.035	1.253	1.190	1.165	0.789	1.523	1.120	1.141	1.421	0.970	0.970	1.217	1.200	1.453	1.175	1.132	1.242	1.014	1.211	0.840	0.973	1.310	1.617
Llama 4 Maverick Instruct	1.271	1.552	1.023	1.260	1.000	0.789	1.651	1.130	1.283	1.684	1.490	1.020	1.735	1.250	0.849	0.990	1.319	1.099	1.606	1.358	0.840	0.699	1.465	2.133
GPT-5 nano (2025-08-07)	1.295	1.290	1.345	1.000	1.319	1.085	1.616	1.310	1.511	1.368	1.340	1.350	1.253	1.440	1.000	0.814	1.341	1.659	1.366	1.232	1.300	1.411	1.521	0.917
Llama 4 Scout Instruct	1.485	1.830	1.241	1.010	1.505	1.070	1.419	1.440	1.370	1.211	1.340	1.500	1.843	1.480	1.340	1.351	2.165	1.418	1.746	1.568	1.030	1.123	1.606	2.550
Qwen2.5-VL Instruct (3B)	1.607	1.443	1.632	1.360	1.593	1.718	1.837	1.430	1.413	1.895	1.430	1.380	1.530	1.420	1.566	1.320	1.769	1.714	1.648	1.453	1.430	1.548	1.690	2.733

Table 10: Full benchmarking results on RoboRewardBench. Models are ranked by mean absolute error (lower is better). The columns report absolute error for each dataset subset.